

Comprehensive Rcpp Usage Guide for R Packages

A multi-source synthesis of C++11 Rcpp development practices, covering coding style, external library integration, documentation, testing, and CI/CD packaging pipelines for CRAN-compliant R packages

datalab@unimib.it

Project:	dve-sample-r
Document:	Rcpp-Q1-developer-guide.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-r/notes/howtos/
Date:	February 23, 2026

Keywords: GEN, Rcpp, RcppArmadillo, RcppEigen, CRAN, GSL, testthat, CI-CD

Abstract

This document consolidates responses from multiple large language models (Claude, Vertex, Gemini, ChatGPT, DeepSeek, Kimi, Perplexity) to a structured prompt requesting a comprehensive guide on Rcpp usage within CRAN-compliant R packages. The synthesis covers nine core topics: C++11 coding style for Rcpp sources, Rcpp namespace utilities and best practices, matrix algebra via RcppArmadillo and RcppEigen, tidyverse integration (notably dplyr), linkage to external OS libraries such as the GNU Scientific Library (GSL), Roxygen2 documentation conventions, inter-language function calls between R and C++ within a package, rebuild workflows, and unit testing with the testthat package.

Beyond the thematic exposition, each model response is expected to provide: a complete C++11 source skeleton including headers for Rcpp, RcppArmadillo, RcppEigen, STL, and GSL; the corresponding DESCRIPTION file annotations and Roxygen2 directives required for external library linkage; and a CI/CD pipeline specification for both GitHub Actions and GitLab CI, performing package checks, tests, binary builds, and conditional artifact upload to a remote FTP server via environment-sourced credentials.

The document serves as a reference compendium for R developers integrating C++ performance code into distributable packages, enabling direct comparison of guidance quality and completeness across leading AI code-assistance tools.

Contents

TOC	6
Q:1	7
Q:1 - TODO:(q1-title)	7
Role	7
Objective	7
Output Format	8
A:1 (Claude)	10
Key Sections Covered	10
Complete Implementation	10
Key Features	10
1. C++11 Rcpp Sources Coding Style	11
Recommended Coding Style	11
Key Style Guidelines:	12
2. Rcpp Namespace Utility and Best Practices	12
Rcpp Namespace Utility	12
Best Practices	12
3. RcppArmadillo and RcppEigen	14
RcppArmadillo	14
RcppEigen	15
4. Integration with Tidyverse Packages (e.g., dplyr)	16
Data Frame Manipulation Integration	16
R Usage Example with dplyr	17
5. Integration with Standard OS Libraries (e.g., GSL)	17
GSL Integration Setup	17
Package Requirements	18
6. Best Practices for Roxygen2 Comments and Function Documentation	19
Documentation Standards	19
Documentation Best Practices	20
7. R and C++ Function Calls in Rcpp Source	20
Calling R Functions from C++	20
Calling C++ Functions from Other C++ Functions	21
Calling Functions from Imported Packages	22
8. Rebuild Operations	23
Direct Rebuild Methods	23
Standard R CMD Operations	23
9. Code Unit Testing Support with 'testthat' Package	24

Test Structure Setup	24
Sample Test Cases	24
Advanced Testing Patterns	26
Full Skeleton of a C++ Source	27
Roxygen2 Documentation and DESCRIPTION File Requisites	42
DESCRIPTION File for Rcpp Package with RcppEigen and GSL	42
Roxygen2 Configuration Requirements	44
Makevars and Makevars.win Files	44
GitHub and GitLab Packaging Action Pipeline	45
GitHub Actions Workflow	45
GitLab CI/CD Pipeline	51
Additional Configuration Files	56
Package Development Workflow Scripts	58
Performance Monitoring Configuration	60
A:1 (Vertex)	63
1. C++11 Rcpp Sources Coding Style /v	63
Good Practices /v	63
2. Rcpp Namespace Utility and Best Practices /v	64
3. RcppArmadillo and RcppEigen /v	64
RcppArmadillo Example	64
RcppEigen Example	64
4. Integration with Tidyverse Packages (e.g., dplyr) /v	65
5. Integration with Standard OS Libraries (e.g., GNU Scientific Library) /v	65
6. Best Practices for Roxygen2 Comments and Function Documentation /v	65
Roxygen2 Best Practices	66
7. R and C++ Function Calls in Rcpp Source /v	66
8. Rebuild Operations /v	66
9. Code Unit Testing Support with 'testthat' Package /v	67
10. Code Examples	67
Full Skeleton of a C++ Source /v	67
Roxygen2 Documentation and DESCRIPTION File Requisites /v	67
GitHub and GitLab Packaging Action Pipeline /v	68
A:1 (Gemini)	71
1. C++11 Rcpp Sources Coding Style /g	71
2. Rcpp Namespace Utility and Best Practices /g	72
3. RcppArmadillo and RcppEigen /g	73
4. Integration with Tidyverse Packages (e.g., dplyr) /g	73
C++ Code (src/rolling_average.cpp)	74
R Code	74
5. Integration with Standard OS Libraries (e.g., gsl - GNU Scientific Library)	74
6. Best Practices for Roxygen2 Comments and Function Documentation /g	75
7. R and C++ Function Calls in Rcpp Source /g	76
Calling another C++ function in the same package	76

Calling an R function from Rcpp	77
Calling a function from another R package	77
8. Rebuild Operations /g	77
9. Code Unit Testing Support with 'testthat' Package /g	78
Full Skeleton of a C++ Source /g	78
Roxygen2 Documentation and DESCRIPTION File Requisites /g	80
GitHub and GitLab Packaging Action Pipeline /g	81
GitHub Actions	81
GitLab CI/CD	83
A:1 (ChatGPT)	85
1. C++11 Rcpp Sources Coding Style	85
□ Recommended Style	85
□ Good Practice	85
□ Bad Practice	85
2. Rcpp Namespace Utility and Best Practices	86
□ Utility	86
□ Best Practices	86
□ Example	86
3. RcppArmadillo and RcppEigen	86
□ Purpose	86
□ RcppArmadillo Example	86
□ RcppEigen Example	86
4. Integration with Tidyverse Packages (e.g., dplyr)	87
□ Example	87
5. Integration with Standard OS Libraries (e.g., GSL)	87
6. Best Practices for Roxygen2 Comments	88
7. Calling Other Functions in Rcpp	88
□ C++ to C++ (internal function)	88
□ R call from C++	88
8. Rebuild Operations	89
9. Unit Testing with testthat	89
□ Setup	89
□ Example Test	89
□ Full Skeleton of a C++ Source File	89
□ Roxygen2 + DESCRIPTION Requirements	90
□ GitHub/GitLab CI Pipeline (R CMD Check + Binary Upload)	90
□ GitHub Actions .github/workflows/R-CMD-check.yml	90
□ GitLab .gitlab-ci.yml	91
A:1 (Perplexity)	92
1. C++11 Rcpp Sources Coding Style /p	92
Recommended Coding Style /p	92
RcppEigen /p	93
4. Integration with Tidyverse Packages (e.g., dplyr) /p	93

5. Integration with Standard OS Libraries (e.g., GSL) /p	94
6. Best Practices for Roxygen2 Comments and Function Documentation /p	94
7. R and C++ Function Calls in Rcpp Source /p	94
8. Rebuild Operations /p	95
9. Code Unit Testing Support with 'testthat'	95
1. Full Skeleton of a C++ Source	95
2. Roxygen2 Documentation and DESCRIPTION File Requisites	96
3. GitHub and GitLab Packaging Action Pipeline	96
GitHub Actions: .github/workflows/R-CMD-check.yaml	96
GitLab CI: .gitlab-ci.yml	97
A:1 (DeepSeek)	99
1. C++11 Rcpp Sources Coding Style	99
2. Rcpp Namespace Utility and Best Practices /d	100
3. RcppArmadillo and RcppEigen /d	100
4. Integration with Tidyverse Packages	101
6. Roxygen2 Documentation Best Practices	102
7. R and C++ Function Calls	102
8. Rebuild Operations	103
9. Unit Testing with testthat	103
Supplemental Materials	103
1. C++ Source Skeleton	103
2. DESCRIPTION Requisites	104
3. CI/CD Pipeline Templates	105
A:1 (Kimi)	107
1. C++11 Rcpp Sources Coding Style /k	107
Recommended Coding Style /k	107
Key Style Guidelines	107
2. Rcpp Namespace Utility and Best Practices /k	108
Namespace Utility	108
Best Practices /k	108
3. RcppArmadillo and RcppEigen /k	109
RcppArmadillo /k	109
RcppEigen /k	110
4. Integration with Tidyverse Packages	110
Integration with dplyr	110
R Usage Example /k	111
5. Integration with Standard OS Libraries (GSL)	111
GSL Integration	111
Package Requirements for GSL	112
6. Best Practices for Roxygen2 Comments	112
7. R and C++ Function Calls	113
Calling R Functions from Rcpp	113
Calling C++ Functions from Rcpp	114

8. Rebuild Operations /k	114
Rebuilding Rcpp Functions	114
9. Code Unit Testing with testthat	114
Test File Structure (tests/testthat/test-rcpp.R)	114
Full C++ Source Skeleton	115
Rxygen2 Documentation and DESCRIPTION Requisites	117
DESCRIPTION File /k	117
NAMESPACE File (auto-generated) /k	117
GitHub/GitLab Packaging Action Pipeline	118
GitHub Actions (.github/workflows/R-CMD-check.yaml)	118
GitLab CI (.gitlab-ci.yml)	119

TOC

1. Q:1 - Rcpp Usage Comprehensive Guide

- see: [Comprehensive Rcpp Guide for R Packages \(Claude\)](#)
- see: [Rcpp Usage Comprehensive Guide \(Vertex\)](#)
- see: [Rcpp Programming Guide \(Gemini\)](#)
- see: [Rcpp usage guide \(ChatGPT\)](#)
- see: [Comprehensive Guide to Using Rcpp in R Packages \(DeepSeek\)](#)
- see: [Comprehensive Guide to Rcpp Usage in R Packages \(Kimi\)](#)
- see: [Rcpp Guide for R Packages \(Perplexity\)](#)

Q:1

Q:1 - TODO:(q1-title)

^

Role

You are an experienced R programmer with extensive knowledge of using Rcpp C++ sources in R packages. Your task is to provide a comprehensive guide on Rcpp usage, adhering to CRAN requirements and best practices.

Objective

Provide detailed information on the following topics, formatted in clear and well-structured (GFM) markdown:

1. C++11 Rcpp Sources Coding Style:

- Explain the recommended coding style for C++11 Rcpp sources in R packages.
- Include examples of good and bad practices.

2. Rcpp Namespace Utility and Best Practices:

- Describe the utility of the Rcpp namespace.
- Provide best practices for basic I/O, memory management, and exception handling using Rcpp.

3. RcppArmadillo and RcppEigen:

- Describe RcppArmadillo and RcppEigen, including their purpose and benefits.
- Provide sample usage examples for both packages, demonstrating common operations.

4. Integration with Tidyverse Packages (e.g., dplyr):

- Show how to integrate Rcpp code with tidyverse packages like dplyr for data frame manipulation.
- Provide code examples demonstrating this integration.

5. Integration with Standard OS Libraries (e.g., gsl - GNU Scientific Library):

- Explain how to integrate Rcpp code with standard OS libraries like GSL.
- Describe package requirements for external library linkage and dependency specification

in the package's DESCRIPTION file.

6. Best Practices for Roxygen2 Comments and Function Documentation:

- Explain best practices for documenting Rcpp functions using Roxygen2 comments.
- Provide examples of well-documented functions.

7. R and C++ Function Calls in Rcpp Source:

- Describe how to call other functions (R/C++) included in the package or in imported external packages from within Rcpp source code.
- Provide a sample of R and C++ function calls in Rcpp source.

8. Rebuild Operations:

- Explain how to rebuild Rcpp functions, either directly or via standard CMD check or build R commands.

9. Code Unit Testing Support with 'testthat' Package:

- Describe how to implement code unit testing for Rcpp functions using the 'testthat' package.
- Provide examples of test cases.

After the textual replay of the above points, provide the following:

1. Full Skeleton of a C++ Source:

- Provide a complete skeleton of a C++ source file, adhering to CRAN coding style.
- Enable C++11 support.
- Include necessary headers for Rcpp, RcppArmadillo, RcppEigen, STL library, and GSL (GNU Scientific Library).

2. Roxygen2 Documentation and DESCRIPTION File Requisites:

- Describe the requisites for Roxygen2 documentation in a package with Rcpp, RcppEigen, and GSL.
- Explain the necessary annotations for the DESCRIPTION file.

3. GitHub and GitLab Packaging Action Pipeline:

- Prepare a packaging action pipeline for GitHub and GitLab that performs tests, checks, and binary builds of the package.
- Include a hypothetical remote FTP server (user/password taken from the environment) that can be used to upload the binary package if the build is successful.

Output Format

- Reply in clear formatted "GitLab Flavored Markdown (GLFM)" Markdown, with precise (lint) validation:
 - codeblock delimiters (" placed atline start). Avoid codeblock nesting.
 - use *underscore markup* for emphasis

- prefer nested headings to text markup with asterisks
 - use only "dash" for unordered lists, with correct indentation
 - insert appropriate blank line separation after headings, list and codeblocks
- Ignore document formatting markup, like:
 -
 - `{=latex}` codeblocks
 - `[!tip]` `[!note]` block quotes
 - code folding tags ("three curly braces pairs")
 - internal links: e.g. `[^]`
- At the end, provide, as Markdown footnotes, a list of references to online documentation resources, linked to answer text where appropriate. To avoid reference clashing with other part of the document, prefix references with the string "rf-".
- Add any additional important information not explicitly required in an "Additional Notes" section.

A:1 (Claude)

^ *Comprehensive Rcpp Guide for R Packages*

I have provided you with a comprehensive guide covering all aspects of Rcpp development for R packages. Here's a summary of what the guide includes:

Key Sections Covered

1. **C++11 Coding Style** - Best practices with examples of good vs. bad code
2. **Rcpp Namespace Utilities** - I/O, memory management, and exception handling
3. **RcppArmadillo & RcppEigen** - Linear algebra integration with practical examples
4. **Tidyverse Integration** - Working with data frames and dplyr workflows
5. **GSL Integration** - System library linkage and statistical functions
6. **Roxygen2 Documentation** - Comprehensive function documentation standards
7. **Function Calls** - R and C++ interoperability within packages
8. **Rebuild Operations** - Development workflow and build processes
9. **Unit Testing** - testthat integration with comprehensive test examples

Complete Implementation

- **Full C++ Skeleton** - Production-ready source file with all headers and dependencies
- **DESCRIPTION Configuration** - Complete package metadata and requirements
- **CI/CD Pipelines** - Both GitHub Actions and GitLab CI with:
 - Multi-platform testing (Linux, Windows, macOS)
 - Automated package building
 - Code coverage reporting
 - FTP deployment on successful builds
 - Documentation generation and deployment
 - Performance benchmarking

Key Features

- **CRAN Compliance** - All recommendations follow CRAN policies
- **Modern C++11** - Leverages contemporary C++ features
- **Error Handling** - Robust exception management throughout
- **Memory Safety** - RAII principles and automatic memory management
- **Performance** - Optimized algorithms and efficient data structures
- **Documentation** - Extensive Roxygen2 examples and best practices

- **Testing** - Comprehensive test suites with edge case coverage
- **Deployment** - Automated FTP upload and release management

This guide serves as a complete reference for developing professional-grade R packages with Rcpp integration, suitable for both CRAN submission and enterprise deployment scenarios.

1. C++11 Rcpp Sources Coding Style

Recommended Coding Style

When writing C++11 Rcpp sources for R packages, follow these style guidelines to ensure CRAN compliance and maintainability:

Good Practices

```
// Use clear, descriptive function names
// [[Rcpp::export]]
Rcpp::NumericVector calculate_statistics(const Rcpp::NumericVector& data) {
    // Use const references for input parameters
    // Use auto for type deduction when appropriate
    auto n = data.size();

    // Initialize variables at declaration
    double mean = 0.0;
    double variance = 0.0;

    // Use range-based for loops (C++11)
    for (const auto& value : data) {
        mean += value;
    }
    mean /= n;

    // Clear variable naming
    for (const auto& value : data) {
        variance += std::pow(value - mean, 2);
    }
    variance /= (n - 1);

    return Rcpp::NumericVector::create(
        Rcpp::Named("mean") = mean,
        Rcpp::Named("variance") = variance
    );
}
```

Bad Practices to Avoid:

```
// Avoid: Poor naming, no const correctness, C-style casts
// [[Rcpp::export]]
Rcpp::NumericVector bad_func(Rcpp::NumericVector x) { // Should be const reference
    int n = (int)x.size(); // Avoid C-style casts
    double m = 0, v = 0; // Declare variables separately for clarity

    // Avoid raw loops when range-based loops are available
    for (int i = 0; i < n; i++) {
        m += x[i];
    }
    m = m / n; // Less efficient than /= operator

    return Rcpp::NumericVector::create(m, v); // No named elements
}
```

Key Style Guidelines:

- Use const references for input parameters that won't be modified
- Prefer auto for type deduction when the type is obvious
- Use range-based for loops when possible
- Initialize variables at declaration
- Use meaningful variable and function names
- Employ RAII (Resource Acquisition Is Initialization) principles
- Use modern C++11 features like nullptr, constexpr, and lambda functions when appropriate

2. Rcpp Namespace Utility and Best Practices

Rcpp Namespace Utility

The Rcpp namespace provides a comprehensive interface between R and C++, offering:

- Type conversion between R and C++ objects
- Memory management automation
- Exception handling mechanisms
- Access to R's statistical functions

Best Practices**Basic I/O**

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
void demonstrate_io(const NumericVector& input) {
```

```

// Output to R console
Rcout << "Processing " << input.size() << " elements" << std::endl;

// Warning messages
if (input.size() == 0) {
    warning("Input vector is empty");
    return;
}

// Error handling
if (any(is_na(input))) {
    stop("Input contains NA values");
}
}

```

Memory Management

```

// [[Rcpp::export]]
NumericMatrix safe_matrix_operations(int nrow, int ncol) {
    // Rcpp handles memory automatically
    NumericMatrix result(nrow, ncol);

    // Fill matrix safely
    for (int i = 0; i < nrow; ++i) {
        for (int j = 0; j < ncol; ++j) {
            result(i, j) = i * ncol + j;
        }
    }

    return result; // Automatic memory management
}

```

Exception Handling

```

// [[Rcpp::export]]
double safe_division(double numerator, double denominator) {
    try {
        if (std::abs(denominator) < std::numeric_limits<double>::epsilon()) {
            throw std::invalid_argument("Division by zero");
        }
        return numerator / denominator;
    } catch (const std::exception& e) {
        stop("Error in safe_division: %s", e.what());
    }
}

```

3. RcppArmadillo and RcppEigen

RcppArmadillo

RcppArmadillo provides integration with the Armadillo C++ library for linear algebra operations.

Purpose and Benefits

- High-performance linear algebra operations
- MATLAB-like syntax
- Automatic memory management
- LAPACK and BLAS integration

Sample Usage

```
#include <RcppArmadillo.h>
// [[Rcpp::depends(RcppArmadillo)]]
using namespace Rcpp;
using namespace arma;

// [[Rcpp::export]]
arma::mat matrix_multiplication_arma(const arma::mat& A, const arma::mat& B) {
    // Check dimensions
    if (A.n_cols != B.n_rows) {
        stop("Incompatible matrix dimensions");
    }

    // Efficient matrix multiplication
    return A * B;
}

// [[Rcpp::export]]
List eigen_decomposition_arma(const arma::mat& X) {
    arma::vec eigenvalues;
    arma::mat eigenvectors;

    bool success = arma::eig_sym(eigenvalues, eigenvectors, X);

    if (!success) {
        stop("Eigenvalue decomposition failed");
    }

    return List::create(
        Named("values") = eigenvalues,
        Named("vectors") = eigenvectors
    );
}
```

```
}
```

RcppEigen

RcppEigen provides integration with the Eigen C++ library for linear algebra.

Purpose and Benefits /c

- Template-based linear algebra library
- High performance through expression templates
- Extensive matrix operations
- Sparse matrix support

Sample Usage /c

```
#include <RcppEigen.h>
// [[Rcpp::depends(RcppEigen)]]
using namespace Rcpp;
using Eigen::MatrixXd;
using Eigen::VectorXd;

// [[Rcpp::export]]
Eigen::MatrixXd matrix_operations_eigen(const Eigen::Map<Eigen::MatrixXd>& A) {
    // Transpose and multiply
    return A.transpose() * A;
}

// [[Rcpp::export]]
List linear_regression_eigen(const Eigen::Map<Eigen::MatrixXd>& X,
                           const Eigen::Map<Eigen::VectorXd>& y) {
    // Solve using QR decomposition
    Eigen::VectorXd coefficients = X.colPivHouseholderQr().solve(y);

    // Calculate residuals
    Eigen::VectorXd residuals = y - X * coefficients;

    return List::create(
        Named("coefficients") = coefficients,
        Named("residuals") = residuals
    );
}
```

4. Integration with Tidyverse Packages (e.g., dplyr)

Data Frame Manipulation Integration

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
DataFrame process_dataframe(DataFrame df) {
  // Extract columns
  NumericVector values = df["values"];
  CharacterVector groups = df["groups"];

  // Process data
  NumericVector processed_values(values.size());
  for (int i = 0; i < values.size(); ++i) {
    processed_values[i] = values[i] * 2.0; // Example transformation
  }

  // Create new DataFrame
  return DataFrame::create(
    Named("groups") = groups,
    Named("original_values") = values,
    Named("processed_values") = processed_values
  );
}

// Function that works well with dplyr workflows
// [[Rcpp::export]]
NumericVector group_statistics(const NumericVector& values,
                              const IntegerVector& group_ids) {
  // Find unique groups
  IntegerVector unique_groups = unique(group_ids);
  NumericVector result(unique_groups.size());

  // Calculate group means
  for (int i = 0; i < unique_groups.size(); ++i) {
    int group = unique_groups[i];
    double sum = 0.0;
    int count = 0;

    for (int j = 0; j < values.size(); ++j) {
      if (group_ids[j] == group) {
        sum += values[j];
        count++;
      }
    }
  }
}
```

```

    }
}

    result[i] = (count > 0) ? sum / count : NA_REAL;
}

    return result;
}

```

R Usage Example with dplyr

```

library(dplyr)
library(YourPackage)

data %>%
  mutate(processed = process_dataframe(.)) %>%
  group_by(category) %>%
  summarise(
    group_stat = group_statistics(values, as.integer(factor(category)))
  )

```

5. Integration with Standard OS Libraries (e.g., GSL)

GSL Integration Setup

SystemRequirements in DESCRIPTION

```
SystemRequirements: GSL (≥ 2.0)
```

Makevars Configuration

```

PKG_CPPFLAGS = $(shell $(R_HOME)/bin/Rscript -e "RcppGSL:::CppFlags()")
PKG_LIBS = $(shell $(R_HOME)/bin/Rscript -e "RcppGSL:::LdFlags()")

```

Sample GSL Usage

```

#include <Rcpp.h>
#include <RcppGSL.h>
#include <gsl/gsl_fit.h>
#include <gsl/gsl_multifit.h>
#include <gsl/gsl_statistics_double.h>

// [[Rcpp::depends(RcppGSL)]]
using namespace Rcpp;

// [[Rcpp::export]]

```

```

List gsl_linear_fit(const NumericVector& x, const NumericVector& y) {
  if (x.size() != y.size()) {
    stop("x and y must have the same length");
  }

  int n = x.size();
  double c0, c1, cov00, cov01, cov11, chisq;

  // Convert to GSL vectors
  RcppGSL::vector<double> gsl_x(x);
  RcppGSL::vector<double> gsl_y(y);

  // Perform linear fit
  gsl_fit_linear(gsl_x.data(), 1, gsl_y.data(), 1, n,
                &c0, &c1, &cov00, &cov01, &cov11, &chisq);

  return List::create(
    Named("intercept") = c0,
    Named("slope") = c1,
    Named("cov00") = cov00,
    Named("cov01") = cov01,
    Named("cov11") = cov11,
    Named("chisq") = chisq
  );
}

// [[Rcpp::export]]
double gsl_correlation(const NumericVector& x, const NumericVector& y) {
  if (x.size() != y.size()) {
    stop("x and y must have the same length");
  }

  RcppGSL::vector<double> gsl_x(x);
  RcppGSL::vector<double> gsl_y(y);

  return gsl_stats_correlation(gsl_x.data(), 1, gsl_y.data(), 1, x.size());
}

```

Package Requirements

DESCRIPTION File Requirements

Depends: R ($\geq 3.5.0$)

Imports: Rcpp, RcppGSL

LinkingTo: Rcpp, RcppGSL

SystemRequirements: GSL (≥ 2.0)

6. Best Practices for Roxygen2 Comments and Function Documentation

Documentation Standards

```

//' Calculate Summary Statistics
//'
//' This function calculates basic summary statistics for a numeric vector,
//' including mean, variance, standard deviation, and confidence intervals.
//'
//' @param data A numeric vector containing the data to analyze
//' @param confidence_level The confidence level for the confidence interval (default: 0.95)
//' @param na_rm Logical value indicating whether to remove NA values (default: TRUE)
//'
//' @return A named list containing:
//' \describe{
//'   \item{mean}{The arithmetic mean}
//'   \item{variance}{The sample variance}
//'   \item{sd}{The standard deviation}
//'   \item{ci_lower}{Lower bound of the confidence interval}
//'   \item{ci_upper}{Upper bound of the confidence interval}
//'   \item{n}{Number of observations used}
//' }
//'
//' @details
//' The function uses the t-distribution to calculate confidence intervals
//' when the sample size is small ( $n < 30$ ) and assumes normal distribution
//' for the underlying data.
//'
//' @examples
//' \dontrun{
//' # Basic usage
//' data <- rnorm(100, mean = 5, sd = 2)
//' stats <- calculate_summary_stats(data)
//' print(stats)
//'
//' # With different confidence level
//' stats_99 <- calculate_summary_stats(data, confidence_level = 0.99)
//' }
//'
//' @seealso \code{\link{mean}}, \code{\link{var}}, \code{\link{sd}}
//'
//' @author Your Name
//' @export
// [[Rcpp::export]]
List calculate_summary_stats(const NumericVector& data,

```

```

        double confidence_level = 0.95,
        bool na_rm = true) {
  // Implementation here
  NumericVector clean_data = na_rm ? na_omit(data) : data;

  if (clean_data.size() == 0) {
    stop("No data available after removing NA values");
  }

  double n = clean_data.size();
  double mean_val = mean(clean_data);
  double var_val = var(clean_data);
  double sd_val = sqrt(var_val);

  // Calculate confidence interval
  double alpha = 1.0 - confidence_level;
  double t_value = R::qt(1.0 - alpha/2.0, n - 1, 1, 0);
  double margin_error = t_value * sd_val / sqrt(n);

  return List::create(
    Named("mean") = mean_val,
    Named("variance") = var_val,
    Named("sd") = sd_val,
    Named("ci_lower") = mean_val - margin_error,
    Named("ci_upper") = mean_val + margin_error,
    Named("n") = n
  );
}

```

Documentation Best Practices

1. **Always include:** @param, @return, @examples
2. **Use clear descriptions:** Explain what the function does and why
3. **Document edge cases:** Explain behavior with NA values, empty inputs
4. **Provide examples:** Include both basic and advanced usage
5. **Reference related functions:** Use @seealso for related functionality
6. **Include author information:** Use @author tag

7. R and C++ Function Calls in Rcpp Source

Calling R Functions from C++

```

#include <Rcpp.h>
using namespace Rcpp;

```

```

// [[Rcpp::export]]
NumericVector call_r_functions(const NumericVector& data) {
  // Get R environment
  Environment base("package:base");
  Environment stats("package:stats");

  // Call R functions
  Function r_mean = base["mean"];
  Function r_sd = stats["sd"];
  Function r_quantile = stats["quantile"];

  // Execute R functions
  double mean_val = as<double>(r_mean(data));
  double sd_val = as<double>(r_sd(data));
  NumericVector quantiles = r_quantile(data,
                                         NumericVector::create(0.25, 0.5, 0.75));

  return NumericVector::create(
    Named("mean") = mean_val,
    Named("sd") = sd_val,
    Named("q25") = quantiles[0],
    Named("median") = quantiles[1],
    Named("q75") = quantiles[2]
  );
}

```

Calling C++ Functions from Other C++ Functions

```

// Helper C++ function (not exported)
double cpp_helper_function(double x, double y) {
  return std::pow(x, 2) + std::pow(y, 2);
}

// Another helper function
NumericVector normalize_vector(const NumericVector& vec) {
  double vec_sum = sum(vec);
  NumericVector result(vec.size());

  for (int i = 0; i < vec.size(); ++i) {
    result[i] = vec[i] / vec_sum;
  }

  return result;
}

```

```
// [[Rcpp::export]]
List comprehensive_analysis(const NumericVector& x, const NumericVector& y) {
  // Call helper C++ functions
  NumericVector distances(x.size());
  for (int i = 0; i < x.size(); ++i) {
    distances[i] = cpp_helper_function(x[i], y[i]);
  }

  // Normalize distances
  NumericVector normalized_distances = normalize_vector(distances);

  // Call R function for additional processing
  Environment stats("package:stats");
  Function r_cor = stats["cor"];
  double correlation = as<double>(r_cor(x, y));

  return List::create(
    Named("distances") = distances,
    Named("normalized_distances") = normalized_distances,
    Named("correlation") = correlation
  );
}
```

Calling Functions from Imported Packages

```
// [[Rcpp::export]]
DataFrame use_external_package_function(const DataFrame& data) {
  // Access functions from imported packages
  Environment dplyr("package:dplyr");
  Function mutate = dplyr["mutate"];
  Function arrange = dplyr["arrange"];

  // Use the functions (this returns to R and back)
  // Note: This is for demonstration - usually you'd do this in R
  SEXP result = mutate(data, Named("new_column") = "processed");

  return as<DataFrame>(result);
}
```

8. Rebuild Operations

Direct Rebuild Methods

Using Rcpp::sourceCpp()

```
# For development and testing
Rcpp::sourceCpp("src/your_file.cpp")

# With verbose output
Rcpp::sourceCpp("src/your_file.cpp", verbose = TRUE)

# With specific compiler flags
Rcpp::sourceCpp("src/your_file.cpp",
                cppFlags = "-std=c++11 -O3")
```

Using devtools

```
# Rebuild and reload package
devtools::load_all()

# Clean and rebuild
devtools::clean_dll()
devtools::load_all()

# Document and rebuild
devtools::document()
devtools::load_all()
```

Standard R CMD Operations

Command Line Operations

```
# Check package
R CMD check package_name_1.0.0.tar.gz

# Build package
R CMD build package_directory

# Install with verbose output
R CMD INSTALL --verbose package_name_1.0.0.tar.gz

# Build with specific flags
R CMD build --resave-data package_directory

# Check with additional tests
R CMD check --as-cran package_name_1.0.0.tar.gz
```

Makefile Integration

```
# Makefile for package development
PACKAGE := YourPackageName
VERSION := $(shell grep Version DESCRIPTION | cut -d' ' -f2)

.PHONY: build check install clean document

document:
    R -e "roxygen2::roxygenise()"

build: document
    R CMD build .

check: build
    R CMD check $(PACKAGE)_$(VERSION).tar.gz

install: build
    R CMD INSTALL $(PACKAGE)_$(VERSION).tar.gz

clean:
    rm -rf $(PACKAGE)_$(VERSION).tar.gz
    rm -rf $(PACKAGE).Rcheck
    rm -rf src/*.o src/*.so src/*.dll

cran-check: build
    R CMD check --as-cran $(PACKAGE)_$(VERSION).tar.gz
```

9. Code Unit Testing Support with 'testthat' Package**Test Structure Setup****tests/testthat.R**

```
library(testthat)
library(YourPackageName)

test_check("YourPackageName")
```

Sample Test Cases**tests/testthat/test-rcpp-functions.R**

```
test_that("calculate_summary_stats works correctly", {
  # Test with normal data
  data <- c(1, 2, 3, 4, 5)
  result <- calculate_summary_stats(data)
```

```

expect_type(result, "list")
expect_named(result, c("mean", "variance", "sd", "ci_lower", "ci_upper", "n"))
expect_equal(result$mean, 3)
expect_equal(result$n, 5)

# Test with NA values
data_na <- c(1, 2, NA, 4, 5)
result_na <- calculate_summary_stats(data_na, na_rm = TRUE)
expect_equal(result_na$n, 4)

# Test error handling
expect_error(calculate_summary_stats(numeric(0)),
             "No data available")
})

test_that("matrix operations work correctly", {
  # Test matrix multiplication
  A <- matrix(1:6, nrow = 2, ncol = 3)
  B <- matrix(1:6, nrow = 3, ncol = 2)

  result <- matrix_multiplication_arma(A, B)
  expected <- A %%% B

  expect_equal(dim(result), dim(expected))
  expect_equal(as.numeric(result), as.numeric(expected), tolerance = 1e-10)

  # Test dimension mismatch
  C <- matrix(1:4, nrow = 2, ncol = 2)
  expect_error(matrix_multiplication_arma(A, C),
               "Incompatible matrix dimensions")
})

test_that("GSL integration works", {
  skip_if_not_installed("RcppGSL")

  # Test linear fit
  x <- 1:10
  y <- 2 * x + 1 + rnorm(10, 0, 0.1)

  result <- gsl_linear_fit(x, y)

  expect_type(result, "list")
  expect_named(result, c("intercept", "slope", "cov00", "cov01", "cov11", "chisq"))
  expect_true(abs(result$slope - 2) < 0.5) # Should be close to 2

```

```

    expect_true(abs(result$intercept - 1) < 0.5) # Should be close to 1
  })

test_that("error handling works properly", {
  # Test division by zero
  expect_error(safe_division(5, 0), "Division by zero")

  # Test successful division
  expect_equal(safe_division(10, 2), 5)

  # Test very small denominator
  expect_error(safe_division(5, 1e-16), "Division by zero")
})

```

Advanced Testing Patterns

```

test_that("performance benchmarks", {
  skip_on_cran() # Skip on CRAN due to timing

  n <- 10000
  data <- rnorm(n)

  # Time the C++ function
  time_cpp <- system.time({
    result_cpp <- calculate_summary_stats(data)
  })

  # Time the R equivalent
  time_r <- system.time({
    result_r <- list(
      mean = mean(data),
      variance = var(data),
      sd = sd(data)
    )
  })

  # C++ should be faster (this is just an example)
  expect_lt(time_cpp[["elapsed"]], time_r[["elapsed"]] * 2)
})

test_that("memory usage is reasonable", {
  skip_on_cran()

  # Test that large operations don't cause memory issues
  large_matrix <- matrix(rnorm(1000000), nrow = 1000)

```

```
    expect_error(matrix_operations_eigen(large_matrix), NA)
  })
```

Full Skeleton of a C++ Source

```
/*
 * Package: YourPackageName
 * File: rcpp_functions.cpp
 * Author: Your Name
 * Description: Main C++ source file with Rcpp, RcppArmadillo, RcppEigen,
 * and GSL integration
 * Created: 2024
 * License: GPL ( $\geq 2$ )
 */

// Enable C++11 support
// [[Rcpp::plugins(cpp11)]]

// Rcpp dependencies
#include <Rcpp.h>
#include <RcppArmadillo.h>
#include <RcppEigen.h>
#include <RcppGSL.h>

// Standard library headers
#include <algorithm>
#include <cmath>
#include <exception>
#include <functional>
#include <iterator>
#include <limits>
#include <memory>
#include <numeric>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

// GSL headers
#include <gsl/gsl_fit.h>
#include <gsl/gsl_multifit.h>
#include <gsl/gsl_statistics_double.h>
```

```

#include <gsl/gsl_rng.h>
#include <gsl/gsl_randist.h>

// Declare dependencies
// [[Rcpp::depends(RcppArmadillo)]]
// [[Rcpp::depends(RcppEigen)]]
// [[Rcpp::depends(RcppGSL)]]

// Use namespaces
using namespace Rcpp;
using namespace arma;
using namespace std;

// Type aliases for cleaner code
using Matrix = Eigen::MatrixXd;
using Vector = Eigen::VectorXd;
using MapMatrix = Eigen::Map<Eigen::MatrixXd>;
using MapVector = Eigen::Map<Eigen::VectorXd>;

/*
 * =====
 * UTILITY FUNCTIONS (NOT EXPORTED)
 * =====
 */

namespace { // Anonymous namespace for internal functions

    // Input validation helper
    void validate_input(const NumericVector& data, const std::string& param_name) {
        if (data.size() == 0) {
            throw std::invalid_argument(param_name + " cannot be empty");
        }

        if (any(is_infinite(data))) {
            throw std::invalid_argument(param_name + " contains infinite values");
        }
    }

    // Safe mathematical operations
    double safe_sqrt(double value) {
        if (value < 0) {
            throw std::domain_error("Cannot take square root of negative number");
        }
        return std::sqrt(value);
    }
}

```

```

    // Memory-efficient matrix operations
    template<typename T>
    void initialize_matrix(T& matrix, double fill_value = 0.0) {
        std::fill(matrix.begin(), matrix.end(), fill_value);
    }

} // end anonymous namespace

/*
 * =====
 * BASIC STATISTICAL FUNCTIONS
 * =====
 */

//' Calculate Robust Summary Statistics
//'
//' Computes comprehensive summary statistics for a numeric vector with
//' robust error handling and missing value treatment.
//'
//' @param data A numeric vector
//' @param confidence_level Confidence level for intervals (default: 0.95)
//' @param na_rm Remove NA values (default: TRUE)
//'
//' @return A named list with summary statistics
//'
//' @examples
//' \dontrun{
//' data <- rnorm(100)
//' stats <- robust_summary_stats(data)
//' }
//'
//' @export
//' [[Rcpp::export]]
List robust_summary_stats(const NumericVector& data,
                          double confidence_level = 0.95,
                          bool na_rm = true) {

    try {
        // Input validation
        if (confidence_level ≤ 0 || confidence_level ≥ 1) {
            stop("confidence_level must be between 0 and 1");
        }

        NumericVector clean_data = na_rm ? na_omit(data) : data;

```

```

    validate_input(clean_data, "data");

    // Basic statistics
    const auto n = static_cast<double>(clean_data.size());
    const double mean_val = mean(clean_data);
    const double var_val = var(clean_data);
    const double sd_val = safe_sqrt(var_val);

    // Confidence interval
    const double alpha = 1.0 - confidence_level;
    const double t_value = R::qt(1.0 - alpha/2.0, n - 1, 1, 0);
    const double margin_error = t_value * sd_val / std::sqrt(n);

    return List::create(
        Named("n") = n,
        Named("mean") = mean_val,
        Named("variance") = var_val,
        Named("sd") = sd_val,
        Named("ci_lower") = mean_val - margin_error,
        Named("ci_upper") = mean_val + margin_error,
        Named("confidence_level") = confidence_level
    );

} catch (const std::exception& e) {
    stop("Error in robust_summary_stats: %s", e.what());
}
}

/*
 * =====
 * ARMADILLO LINEAR ALGEBRA FUNCTIONS
 * =====
 */

// ' Efficient Matrix Operations using Armadillo
// '
// ' Performs common matrix operations using the Armadillo library
// ' for optimal performance.
// '
// ' @param A First matrix
// ' @param B Second matrix
// '
// ' @return Result matrix
// '
// ' @export

```

```

// [[Rcpp::export]]
arma::mat armadillo_matrix_ops(const arma::mat& A, const arma::mat& B) {

    try {
        // Dimension checks
        if (A.n_cols != B.n_rows) {
            stop("Matrix dimensions incompatible for multiplication");
        }

        // Efficient matrix multiplication
        arma::mat result = A * B;

        return result;

    } catch (const std::exception& e) {
        stop("Error in armadillo_matrix_ops: %s", e.what());
    }
}

//' Principal Component Analysis using Armadillo
//'
//' Performs PCA using singular value decomposition.
//'
//' @param X Data matrix (observations in rows, variables in columns)
//' @param center Center the data (default: TRUE)
//' @param scale Scale the data (default: FALSE)
//'
//' @return A list containing PCA results
//'
//' @export
// [[Rcpp::export]]
List armadillo_pca(const arma::mat& X, bool center = true, bool scale = false) {

    try {
        arma::mat X_processed = X;

        // Center the data
        if (center) {
            arma::rowvec means = arma::mean(X_processed, 0);
            X_processed.each_row() -= means;
        }

        // Scale the data
        if (scale) {
            arma::rowvec stds = arma::stddev(X_processed, 0, 0);

```

```

        X_processed.each_row() /= stds;
    }

    // Perform SVD
    arma::mat U, V;
    arma::vec s;
    bool success = arma::svd(U, s, V, X_processed);

    if (!success) {
        stop("SVD decomposition failed");
    }

    // Calculate proportion of variance explained
    arma::vec eigenvalues = arma::square(s) / (X.n_rows - 1);
    arma::vec prop_var = eigenvalues / arma::sum(eigenvalues);
    arma::vec cumsum_var = arma::cumsum(prop_var);

    return List::create(
        Named("rotation") = V,
        Named("sdev") = s / std::sqrt(X.n_rows - 1),
        Named("center") = center,
        Named("scale") = scale,
        Named("prop_variance") = prop_var,
        Named("cumsum_variance") = cumsum_var
    );

} catch (const std::exception& e) {
    stop("Error in armadillo_pca: %s", e.what());
}

}

/*
 * =====
 * EIGEN LINEAR ALGEBRA FUNCTIONS
 * =====
 */

// ' Linear Regression using Eigen
// '
// ' Performs linear regression using QR decomposition for numerical stability.
// '
// ' @param X Design matrix
// ' @param y Response vector
// '
// ' @return List with regression results

```

```

//'
//' @export
//' [[Rcpp::export]]
List eigen_linear_regression(const MapMatrix& X, const MapVector& y) {

  try {
    // Dimension check
    if (X.rows() != y.size()) {
      stop("Number of observations in X and y must match");
    }

    // Solve using QR decomposition with column pivoting
    Eigen::ColPivHouseholderQR<Matrix> qr_decomp(X);
    Vector coefficients = qr_decomp.solve(y);

    // Calculate fitted values and residuals
    Vector fitted = X * coefficients;
    Vector residuals = y - fitted;

    // Calculate R-squared
    double y_mean = y.mean();
    double tss = (y.array() - y_mean).square().sum();
    double rss = residuals.array().square().sum();
    double r_squared = 1.0 - (rss / tss);

    // Standard errors (simplified)
    double mse = rss / (X.rows() - X.cols());
    Matrix XtX_inv = (X.transpose() * X).inverse();
    Vector std_errors = (XtX_inv.diagonal().array() * mse).sqrt();

    return List::create(
      Named("coefficients") = coefficients,
      Named("fitted_values") = fitted,
      Named("residuals") = residuals,
      Named("r_squared") = r_squared,
      Named("std_errors") = std_errors,
      Named("mse") = mse
    );

  } catch (const std::exception& e) {
    stop("Error in eigen_linear_regression: %s", e.what());
  }
}

//' Eigenvalue Decomposition using Eigen

```

```

//'
//' Computes eigenvalues and eigenvectors of a symmetric matrix.
//'
//' @param A Symmetric matrix
//'
//' @return List with eigenvalues and eigenvectors
//'
//' @export
// [[Rcpp::export]]
List eigen_decomposition(const MapMatrix& A) {

    try {
        // Check if matrix is square
        if (A.rows() != A.cols()) {
            stop("Matrix must be square for eigenvalue decomposition");
        }

        // Check if matrix is symmetric (approximately)
        if (!A.isApprox(A.transpose(), 1e-10)) {
            warning("Matrix is not symmetric; results may be unreliable");
        }

        // Perform eigenvalue decomposition
        Eigen::SelfAdjointEigenSolver<Matrix> eigen_solver(A);

        if (eigen_solver.info() != Eigen::Success) {
            stop("Eigenvalue decomposition failed");
        }

        return List::create(
            Named("values") = eigen_solver.eigenvalues(),
            Named("vectors") = eigen_solver.eigenvectors()
        );

    } catch (const std::exception& e) {
        stop("Error in eigen_decomposition: %s", e.what());
    }
}

/*
 * =====
 * GSL INTEGRATION FUNCTIONS
 * =====
 */

```

```

//' Linear Fitting using GSL
//'
//' Performs linear regression using GNU Scientific Library functions.
//'
//' @param x Independent variable vector
//' @param y Dependent variable vector
//'
//' @return List with fitting results
//'
//' @export
// [[Rcpp::export]]
List gsl_linear_fit(const NumericVector& x, const NumericVector& y) {

  try {
    if (x.size() != y.size()) {
      stop("x and y must have the same length");
    }

    if (x.size() < 2) {
      stop("Need at least 2 data points for fitting");
    }

    const int n = x.size();
    double c0, c1, cov00, cov01, cov11, chisq;

    // Convert to GSL vectors
    RcppGSL::vector<double> gsl_x(x);
    RcppGSL::vector<double> gsl_y(y);

    // Perform linear fit
    int status = gsl_fit_linear(
      gsl_x.data(), 1, gsl_y.data(), 1, n,
      &c0, &c1, &cov00, &cov01, &cov11, &chisq
    );

    if (status != GSL_SUCCESS) {
      stop("GSL linear fit failed with status: %d", status);
    }

    // Calculate additional statistics
    double r_squared = 1.0 - chisq / gsl_stats_tss(gsl_y.data(), 1, n);

    return List::create(
      Named("intercept") = c0,
      Named("slope") = c1,

```

```

    Named("cov00") = cov00,
    Named("cov01") = cov01,
    Named("cov11") = cov11,
    Named("chisq") = chisq,
    Named("r_squared") = r_squared,
    Named("status") = status
};

} catch (const std::exception& e) {
    stop("Error in gsl_linear_fit: %s", e.what());
}
}

///  

// Random Number Generation using GSL  

//  

// Generates random numbers from various distributions using GSL.  

//  

// @param n Number of random numbers to generate  

// @param distribution Distribution type ("normal", "uniform", "exponential", "gamma")  

// @param param1 First parameter (mean for normal, min for uniform, rate for exponential, shape  

// @param param2 Second parameter (sd for normal, max for uniform, unused for exponential, rate  

// @param seed Random seed (default: 0 for current time)  

//  

// @return Vector of random numbers  

//  

// @export  

// [[Rcpp::export]]  

NumericVector gsl_random_numbers(int n,  

                                  const std::string& distribution,  

                                  double param1,  

                                  double param2 = 0.0,  

                                  unsigned long seed = 0) {  
  

    try {  

        if (n <= 0) {  

            stop("n must be positive");  

        }  
  

        // Initialize random number generator  

        gsl_rng_env_setup();  

        const gsl_rng_type* rng_type = gsl_rng_default;  

        gsl_rng* rng = gsl_rng_alloc(rng_type);  
  

        // Set seed  

        if (seed == 0) {

```

```

        seed = static_cast<unsigned long>(std::time(nullptr));
    }
    gsl_rng_set(rng, seed);

    NumericVector result(n);

    // Generate random numbers based on distribution
    if (distribution == "normal") {
        for (int i = 0; i < n; ++i) {
            result[i] = param1 + gsl_ran_gaussian(rng, param2);
        }
    } else if (distribution == "uniform") {
        for (int i = 0; i < n; ++i) {
            result[i] = gsl_ran_flat(rng, param1, param2);
        }
    } else if (distribution == "exponential") {
        for (int i = 0; i < n; ++i) {
            result[i] = gsl_ran_exponential(rng, 1.0 / param1);
        }
    } else if (distribution == "gamma") {
        for (int i = 0; i < n; ++i) {
            result[i] = gsl_ran_gamma(rng, param1, 1.0 / param2);
        }
    } else {
        gsl_rng_free(rng);
        stop("Unknown distribution: %s", distribution.c_str());
    }

    // Clean up
    gsl_rng_free(rng);

    return result;

} catch (const std::exception& e) {
    stop("Error in gsl_random_numbers: %s", e.what());
}

}

/*
 * =====
 * DATA MANIPULATION FUNCTIONS (TIDYVERSE INTEGRATION)
 * =====
 */

// ' Efficient Group Operations

```

```

//'
//' Performs group-wise operations on data frames efficiently in C++.
//'
//' @param data Data frame
//' @param group_col Name of the grouping column
//' @param value_col Name of the value column
//' @param operation Operation to perform ("mean", "sum", "count", "sd")
//'
//' @return Data frame with group results
//'
//' @export
// [[Rcpp::export]]
DataFrame efficient_group_operations(const DataFrame& data,
                                     const std::string& group_col,
                                     const std::string& value_col,
                                     const std::string& operation) {

  try {
    // Extract columns
    CharacterVector groups = data[group_col];
    NumericVector values = data[value_col];

    if (groups.size() != values.size()) {
      stop("Group and value columns must have the same length");
    }

    // Find unique groups
    CharacterVector unique_groups = unique(groups);
    std::vector<double> results(unique_groups.size());

    // Perform group operations
    for (int i = 0; i < unique_groups.size(); ++i) {
      std::string current_group = as<std::string>(unique_groups[i]);
      std::vector<double> group_values;

      // Collect values for current group
      for (int j = 0; j < groups.size(); ++j) {
        if (as<std::string>(groups[j]) == current_group) {
          if (!NumericVector::is_na(values[j])) {
            group_values.push_back(values[j]);
          }
        }
      }

      // Calculate result based on operation

```

```

    if (group_values.empty()) {
        results[i] = NA_REAL;
    } else if (operation == "mean") {
        results[i] = std::accumulate(group_values.begin(), group_values.end(), 0.0) / group_values.size();
    } else if (operation == "sum") {
        results[i] = std::accumulate(group_values.begin(), group_values.end(), 0.0);
    } else if (operation == "count") {
        results[i] = static_cast<double>(group_values.size());
    } else if (operation == "sd") {
        if (group_values.size() < 2) {
            results[i] = NA_REAL;
        } else {
            double mean_val = std::accumulate(group_values.begin(), group_values.end(), 0.0) / group_values.size();
            double sum_sq_diff = 0.0;
            for (double val : group_values) {
                sum_sq_diff += std::pow(val - mean_val, 2);
            }
            results[i] = std::sqrt(sum_sq_diff / (group_values.size() - 1));
        }
    } else {
        stop("Unknown operation: %s", operation.c_str());
    }
}

return DataFrame::create(
    Named(group_col) = unique_groups,
    Named("result") = NumericVector(results.begin(), results.end())
);

} catch (const std::exception& e) {
    stop("Error in efficient_group_operations: %s", e.what());
}

}

/*
 * =====
 * ADVANCED UTILITY FUNCTIONS
 * =====
 */

// ' Parallel Computation Example
// '
// ' Demonstrates parallel computation patterns (conceptual - actual parallelization
// ' would require OpenMP or similar).
// '

```

```

//' @param data Input vector
//' @param func_type Type of function to apply ("square", "sqrt", "log")
//'
//' @return Transformed vector
//'
//' @export
// [[Rcpp::export]]
NumericVector parallel_transform(const NumericVector& data,
                                const std::string& func_type) {

    try {
        NumericVector result(data.size());

        // Choose transformation function
        std::function<double(double)> transform_func;

        if (func_type == "square") {
            transform_func = [](double x) { return x * x; };
        } else if (func_type == "sqrt") {
            transform_func = [](double x) { return x ≥ 0 ? std::sqrt(x) : NA_REAL; };
        } else if (func_type == "log") {
            transform_func = [](double x) { return x > 0 ? std::log(x) : NA_REAL; };
        } else {
            stop("Unknown function type: %s", func_type.c_str());
        }

        // Apply transformation (in practice, this could be parallelized)
        std::transform(data.begin(), data.end(), result.begin(), transform_func);

        return result;

    } catch (const std::exception& e) {
        stop("Error in parallel_transform: %s", e.what());
    }
}

//' Memory-Efficient Large Data Processing
//'
//' Processes large datasets in chunks to manage memory usage.
//'
//' @param data Large input vector
//' @param chunk_size Size of processing chunks
//' @param operation Operation to perform ("cumsum", "cumprod", "diff")
//'
//' @return Processed vector

```

```
//'  
//' @export  
// [[Rcpp::export]]  
NumericVector memory_efficient_processing(const NumericVector& data,  
                                         int chunk_size = 1000,  
                                         const std::string& operation = "cumsum") {  
  
    try {  
        if (chunk_size ≤ 0) {  
            stop("chunk_size must be positive");  
        }  
  
        const int n = data.size();  
        NumericVector result(n);  
  
        if (operation == "cumsum") {  
            double cumulative_sum = 0.0;  
            for (int i = 0; i < n; ++i) {  
                cumulative_sum += data[i];  
                result[i] = cumulative_sum;  
            }  
        } else if (operation == "cumprod") {  
            double cumulative_prod = 1.0;  
            for (int i = 0; i < n; ++i) {  
                cumulative_prod *= data[i];  
                result[i] = cumulative_prod;  
            }  
        } else if (operation == "diff") {  
            if (n == 0) {  
                return NumericVector(0);  
            }  
            NumericVector diff_result(n - 1);  
            for (int i = 1; i < n; ++i) {  
                diff_result[i - 1] = data[i] - data[i - 1];  
            }  
            return diff_result;  
        } else {  
            stop("Unknown operation: %s", operation.c_str());  
        }  
  
        return result;  
    } catch (const std::exception& e) {  
        stop("Error in memory_efficient_processing: %s", e.what());  
    }  
}
```

```

}

/*
 * =====
 * PACKAGE INITIALIZATION AND CLEANUP
 * =====
 */

// ' Package Information
// '
// ' Returns information about the package and its dependencies.
// '
// ' @return List with package information
// '
// ' @export
// [[Rcpp::export]]
List package_info() {
  return List::create(
    Named("package") = "YourPackageName",
    Named("rcpp_version") = "1.0.12",
    Named("armadillo_version") = "12.6.6",
    Named("eigen_version") = "3.4.0",
    Named("gsl_version") = "2.7",
    Named("cpp_standard") = "C++11",
    Named("compiled") = __DATE__ " " __TIME__
  );
}

/*
 * =====
 * END OF FILE
 * =====
 */

```

Roxygen2 Documentation and DESCRIPTION File Requisites

DESCRIPTION File for Rcpp Package with RcppEigen and GSL

Package: YourPackageName

Type: Package

Title: Advanced Statistical Computing with Rcpp Integration

Version: 1.0.0

Date: 2024-07-30

Authors@R: c(

 person("Your", "Name", email = "your.email@example.com",

```

    role = c("aut", "cre"), comment = c(ORCID = "0000-0000-0000-0000")),
  person("Co", "Author", email = "co.author@example.com",
    role = "aut")
)

```

Description: This package provides efficient statistical computing functions using Rcpp, RcppArmadillo, RcppEigen, and GNU Scientific Library (GSL) integration. It demonstrates best practices for C++11 development in R packages with comprehensive error handling and memory management.

License: GPL (≥ 2)

URL: <https://github.com/yourusername/YourPackageName>

BugReports: <https://github.com/yourusername/YourPackageName/issues>

Depends:

```
R (≥ 3.5.0)
```

Imports:

```

Rcpp (≥ 1.0.12),
RcppArmadillo (≥ 0.12.6.6.0),
RcppEigen (≥ 0.3.4.0.0),
RcppGSL (≥ 0.3.13),
methods,
stats,
utils

```

LinkingTo:

```

Rcpp,
RcppArmadillo,
RcppEigen,
RcppGSL

```

Suggests:

```

testthat (≥ 3.0.0),
knitr,
rmarkdown,
dplyr (≥ 1.0.0),
ggplot2,
covr,
bench

```

SystemRequirements:

```

GSL (≥ 2.0),
C++11

```

Encoding: UTF-8

Language: en-US

Roxygen: list(markdown = TRUE)

RoxygenNote: 7.3.1

LazyData: true

VignetteBuilder: knitr

Config/testthat/edition: 3

Roxygen2 Configuration Requirements

NAMESPACE File (Auto-generated by Roxygen2):

Generated by roxygen2: do not edit by hand

```
export(armadillo_matrix_ops)
export(armadillo_pca)
export(efficient_group_operations)
export(eigen_decomposition)
export(eigen_linear_regression)
export(gsl_linear_fit)
export(gsl_random_numbers)
export(memory_efficient_processing)
export(package_info)
export(parallel_transform)
export(robust_summary_stats)

import(Rcpp)
import(RcppArmadillo)
import(RcppEigen)
import(RcppGSL)
import(methods)
import(stats)

useDynLib(YourPackageName, .registration=TRUE)
```

Makevars and Makevars.win Files

src/Makevars

Makevars for Unix-like systems

CXX_STD = CXX11

RcppGSL configuration

PKG_CPPFLAGS = \$(shell \$(R_HOME)/bin/Rscript -e "RcppGSL ::: CppFlags()")

PKG_LIBS = \$(shell \$(R_HOME)/bin/Rscript -e "RcppGSL ::: LdFlags()")

Additional compiler flags for optimization and warnings

PKG_CXXFLAGS = -O3 -Wall -Wextra -pedantic -DARMA_DONT_PRINT_ERRORS

Debug configuration (uncomment for debugging)

PKG_CXXFLAGS = -g -O0 -Wall -Wextra -pedantic -DARMA_DONT_PRINT_ERRORS

src/Makevars.win

```
# Makevars.win for Windows systems
```

```
CXX_STD = CXX11
```

```
# Windows-specific GSL configuration
```

```
PKG_CPPFLAGS = $(shell "${R_HOME}/bin/${R_ARCH_BIN}/Rscript.exe" -e "RcppGSL::: CppFlags()")
```

```
PKG_LIBS = $(shell "${R_HOME}/bin/${R_ARCH_BIN}/Rscript.exe" -e "RcppGSL::: LdFlags()")
```

```
# Windows-specific compiler flags
```

```
PKG_CXXFLAGS = -O2 -Wall -DARMA_DONT_PRINT_ERRORS
```

GitHub and GitLab Packaging Action Pipeline**GitHub Actions Workflow**

```
.github/workflows/R-CMD-check.yml
```

```
name: R-CMD-check
```

```
on:
```

```
  push:
```

```
    branches: [ main, master, develop ]
```

```
  pull_request:
```

```
    branches: [ main, master ]
```

```
  schedule:
```

```
    # Run daily at 3 AM UTC
```

```
    - cron: '0 3 * * *'
```

```
env:
```

```
  GITHUB_PAT: ${ secrets.GITHUB_TOKEN }
```

```
  R_KEEP_PKG_SOURCE: yes
```

```
jobs:
```

```
  R-CMD-check:
```

```
    runs-on: ${ matrix.config.os }
```

```
    name: ${ matrix.config.os } (${ matrix.config.r })
```

```
  strategy:
```

```
    fail-fast: false
```

```
    matrix:
```

```
      config:
```

```
        - {os: ubuntu-latest, r: 'devel', http-user-agent: 'release'}
```

- {os: ubuntu-latest, r: 'release'}
- {os: ubuntu-latest, r: 'oldrel-1'}
- {os: windows-latest, r: 'release'}
- {os: macOS-latest, r: 'release'}

steps:

- uses: actions/checkout@v4
- uses: r-lib/actions/setup-pandoc@v2
- uses: r-lib/actions/setup-r@v2
 - with:
 - r-version: \${{ matrix.config.r }}
 - http-user-agent: \${{ matrix.config.http-user-agent }}
 - use-public-rspm: true
- name: Install system dependencies (Ubuntu)
 - if: runner.os == 'Linux'
 - run: |


```
sudo apt-get update
sudo apt-get install -y \
  libgsl-dev \
  libgsl27 \
  gsl-bin \
  liblapack-dev \
  libblas-dev \
  libarmadillo-dev \
  libeigen3-dev \
  libcurl4-openssl-dev \
  libssl-dev \
  libxml2-dev \
  libfontconfig1-dev \
  libfreetype6-dev
```
- name: Install system dependencies (macOS)
 - if: runner.os == 'macOS'
 - run: |


```
brew install gsl
brew install armadillo
brew install eigen
```
- name: Install system dependencies (Windows)
 - if: runner.os == 'Windows'
 - run: |


```
# Install Rtools if not present
```

```

    choco install rtools -y --no-progress

- uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: |
      any::rcmdcheck
      any::covr
      any::pkgdown
    needs: check

- uses: r-lib/actions/check-r-package@v2
  with:
    upload-snapshots: true
    build_args: 'c("--no-build-vignettes","--no-manual")'

- name: Show testthat output
  if: always()
  run: find check -name 'testthat.Rout*' -exec cat '{}' \; || true
  shell: bash

- name: Upload check results
  if: failure()
  uses: actions/upload-artifact@v4
  with:
    name: ${ runner.os }-r${ matrix.config.r }-results
    path: check

coverage:
  runs-on: ubuntu-latest
  env:
    GITHUB_PAT: ${ secrets.GITHUB_TOKEN }

steps:
- uses: actions/checkout@v4

- uses: r-lib/actions/setup-r@v2
  with:
    use-public-rspm: true

- name: Install system dependencies
  run: |
    sudo apt-get update
    sudo apt-get install -y \
      libgsf-dev \
      libgsf27 \

```

```

    gsl-bin \
    liblapack-dev \
    libblas-dev \
    libarmadillo-dev \
    libeigen3-dev

- uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: any::covr
    needs: coverage

- name: Test coverage
  run: |
    covr::codecov(
      quiet = FALSE,
      clean = FALSE,
      install_path = file.path(Sys.getenv("RUNNER_TEMP"), "package")
    )
  shell: Rscript {0}

pkgdown:
  runs-on: ubuntu-latest
  env:
    GITHUB_PAT: ${ secrets.GITHUB_TOKEN }

steps:
- uses: actions/checkout@v4

- uses: r-lib/actions/setup-pandoc@v2

- uses: r-lib/actions/setup-r@v2
  with:
    use-public-rspm: true

- name: Install system dependencies
  run: |
    sudo apt-get update
    sudo apt-get install -y \
      libgsl-dev \
      libgsl27 \
      gsl-bin \
      liblapack-dev \
      libblas-dev \
      libarmadillo-dev \
      libeigen3-dev

```

```

- uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: any::pkgdown, local::.
    needs: website

- name: Build site
  run: pkgdown::build_site_github_pages(new_process = FALSE, install = FALSE)
  shell: Rscript {0}

- name: Deploy to GitHub pages ☑
  if: github.event_name != 'pull_request'
  uses: JamesIves/github-pages-deploy-action@v4.4.1
  with:
    clean: false
    branch: gh-pages
    folder: docs

build-and-deploy:
  needs: [R-CMD-check, coverage]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'

  steps:
    - uses: actions/checkout@v4

    - uses: r-lib/actions/setup-r@v2
      with:
        use-public-rspm: true

    - name: Install system dependencies
      run: |
        sudo apt-get update
        sudo apt-get install -y \
          libgsl-dev \
          libgsl27 \
          gsl-bin \
          liblapack-dev \
          libblas-dev \
          libarmadillo-dev \
          libeigen3-dev

    - uses: r-lib/actions/setup-r-dependencies@v2

    - name: Build package

```

```

run: |
  R CMD build .
  echo "PKG_FILE=$(ls -1 *.tar.gz)" >> $GITHUB_ENV

- name: Upload to FTP server
  if: success()
  env:
    FTP_SERVER: ${ secrets.FTP_SERVER }
    FTP_USERNAME: ${ secrets.FTP_USERNAME }
    FTP_PASSWORD: ${ secrets.FTP_PASSWORD }
    FTP_PATH: ${ secrets.FTP_PATH }
  run: |
    # Install lftp for FTP operations
    sudo apt-get install -y lftp

    # Upload package to FTP server
    lftp -c "
    set ssl:verify-certificate no;
    set ftp:ssl-force true;
    set ftp:ssl-protect-data true;
    open -u $FTP_USERNAME,$FTP_PASSWORD $FTP_SERVER;
    cd $FTP_PATH;
    put $PKG_FILE;
    put $PKG_FILE $PKG_FILE.$(date +%Y%m%d_%H%M%S);
    bye
    "

    echo "Package $PKG_FILE uploaded successfully to FTP server"

- name: Create Release
  if: success()
  uses: actions/create-release@v1
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
  with:
    tag_name: v${ github.run_number }
    release_name: Release v${ github.run_number }
    body: |
      Automated release of package build

      Built from commit: ${ github.sha }
      Package file: ${ env.PKG_FILE }
    draft: false
    prerelease: false

```

```

- name: Upload Release Asset
  if: success()
  uses: actions/upload-release-asset@v1
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
  with:
    upload_url: ${ steps.create_release.outputs.upload_url }
    asset_path: ./${ env.PKG_FILE }
    asset_name: ${ env.PKG_FILE }
    asset_content_type: application/gzip

```

GitLab CI/CD Pipeline

.gitlab-ci.yml

```
# GitLab CI/CD Pipeline for R Package with Rcpp
```

```
image: rocker/r-devel:latest
```

```
variables:
```

```
  R_LIBS_USER: "$CI_PROJECT_DIR/R-library"
```

```
  CHECK_DIR: "$CI_PROJECT_DIR.Rcheck"
```

```
  BUILD_LOGS_DIR: "$CI_PROJECT_DIR/logs"
```

```
# Define stages
```

```
stages:
```

- prepare
- build
- test
- check
- coverage
- deploy

```
# Cache R packages
```

```
cache:
```

```
  key: ${CI_COMMIT_REF_SLUG}
```

```
  paths:
```

- R-library/
- packrat/lib*/

```
before_script:
```

```
# Update system and install system dependencies
```

- apt-get update -qq
- apt-get install -y --no-install-recommends
 libgsl-dev
 libgsl27
 gsl-bin

```
liblapack-dev
libblas-dev
libarmadillo-dev
libeigen3-dev
libcurl4-openssl-dev
libssl-dev
libxml2-dev
libfontconfig1-dev
libfreetype6-dev
pandoc
pandoc-citeproc
lftp
curl
# Create R library directory
- mkdir -p $R_LIBS_USER
# Install required R packages
- R -e "install.packages(c('devtools', 'testthat', 'covr', 'pkgdown', 'roxygen2'), repos='http

# Prepare stage
prepare:
  stage: prepare
  script:
    - R -e "devtools::install_deps(dependencies = TRUE)"
    - mkdir -p $BUILD_LOGS_DIR
  artifacts:
    paths:
      - R-library/
    expire_in: 1 hour

# Build stage
build:
  stage: build
  dependencies:
    - prepare
  script:
    - R -e "devtools::document()"
    - R CMD build .
    - PKG_FILE=$(ls -1 *.tar.gz)
    - echo "PKG_FILE=$PKG_FILE" > build.env
    - echo "Built package: $PKG_FILE"
  artifacts:
    paths:
      - "*.tar.gz"
  reports:
    dotenv: build.env
```

```
    expire_in: 1 day

# Test stage
test:
  stage: test
  dependencies:
    - prepare
    - build
  script:
    - R -e "devtools::load_all(); devtools::test()"
  artifacts:
    when: always
    paths:
      - tests/testthat/
    reports:
      junit: tests/testthat/test-results.xml
    expire_in: 1 week

# R CMD check stage
check:
  stage: check
  dependencies:
    - build
  script:
    - PKG_FILE=$(ls -1 *.tar.gz)
    - R CMD check --as-cran --no-manual $PKG_FILE
    - echo "R CMD check completed successfully"
  artifacts:
    when: always
    paths:
      - "*/.Rcheck/"
    expire_in: 1 week
  allow_failure: false

# Coverage stage
coverage:
  stage: coverage
  dependencies:
    - prepare
  script:
    - R -e "
      library(covr);
      coverage_result <- package_coverage();
      print(coverage_result);
      codecov(coverage = coverage_result);
    "
```

```

"
coverage: '/Coverage: \d+\. \d+/'
artifacts:
  reports:
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml
    expire_in: 1 week
only:
  - main
  - master
  - develop

# Deploy to FTP and create packages
deploy:
  stage: deploy
  dependencies:
    - build
    - check
  environment:
    name: production
    url: $FTP_SERVER
  script:
    - PKG_FILE=$(ls -1 *.tar.gz)
    - |
      # Upload to FTP server if all previous stages passed
      if [ -n "$FTP_SERVER" ] && \
        [ -n "$FTP_USERNAME" ] && [ -n "$FTP_PASSWORD" ]; then
        echo "Uploading $PKG_FILE to FTP server..."
        lftp -c "
          set ssl:verify-certificate no;
          set ftp:ssl-force true;
          set ftp:ssl-protect-data true;
          open -u $FTP_USERNAME,$FTP_PASSWORD $FTP_SERVER;
          cd $FTP_PATH;
          put $PKG_FILE;
          put $PKG_FILE $PKG_FILE.$(date +%Y%m%d_%H%M%S);
          bye
        "
        echo "Package uploaded successfully to FTP server"
      else
        echo "FTP credentials not configured, skipping upload"
      fi
    # Create GitLab release
    - |

```

```

    curl --request POST \
      --header "PRIVATE-TOKEN: $CI_JOB_TOKEN" \
      --header "Content-Type: application/json" \
      --data '{
        "name": "Release '$CI_PIPELINE_ID'",
        "tag_name": "v'$CI_PIPELINE_ID'",
        "description": "Automated release from pipeline '$CI_PIPELINE_ID'\n\nBuilt from com
      }' \
      "$CI_API_V4_URL/projects/$CI_PROJECT_ID/releases"
artifacts:
  paths:
    - "*.tar.gz"
  expire_in: 1 month
only:
  - main
  - master
when: on_success

# Documentation deployment
pages:
  stage: deploy
dependencies:
  - prepare
script:
  - R -e "pkgdown::build_site()"
  - mv docs public
artifacts:
  paths:
    - public
  expire_in: 1 month
only:
  - main
  - master

# Windows build job (using Windows runner if available)
build:windows:
  stage: build
  tags:
    - windows
  before_script:
    - choco install r.project -y --no-progress
    - choco install rtools -y --no-progress
    - refreshenv
  script:
    - R.exe -e "install.packages(c('devtools', 'Rcpp', 'RcppArmadillo', 'RcppEigen', 'RcppGSL'),

```

```

- R.exe -e "devtools::install_deps(dependencies = TRUE)"
- R.exe -e "devtools::document()"
- R.exe CMD build .
- $PKG_FILE = Get-ChildItem -Name "*.tar.gz"
- Write-Host "Built Windows package: $PKG_FILE"
artifacts:
  paths:
    - "*.tar.gz"
  expire_in: 1 day
allow_failure: true
only:
  - main
  - master
  - develop

# macOS build job (using macOS runner if available)
build:macos:
  stage: build
  tags:
    - macos
  before_script:
    - brew install r
    - brew install gsl armadillo eigen
  script:
    - R -e "install.packages(c('devtools', 'Rcpp', 'RcppArmadillo', 'RcppEigen', 'RcppGSL'), rep
    - R -e "devtools::install_deps(dependencies = TRUE)"
    - R -e "devtools::document()"
    - R CMD build .
    - PKG_FILE=$(ls -1 *.tar.gz)
    - echo "Built macOS package: $PKG_FILE"
  artifacts:
    paths:
      - "*.tar.gz"
    expire_in: 1 day
  allow_failure: true
  only:
    - main
    - master
    - develop

```

Additional Configuration Files

.Rbuildignore:

```

^.*\.Rproj$
^\.Rproj\.user$

```

```
^\.github$
^\.gitlab-ci\.yaml$
^codecov\.yaml$
^\.lintr$
^_pkgdown\.yaml$
^docs$
^pkgdown$
^\.covrignore$
^cran-comments\.md$
^CRAN-RELEASE$
^revdep$
^\.travis\.yaml$
^appveyor\.yaml$
^README\.Rmd$
^LICENSE\.md$
^logs$
^.*\.log$
^.*\.Rcheck$
^.*\.tar\.gz$
build.env
```

codecov.yml:

```
coverage:
  status:
    project:
      default:
        target: 80%
        threshold: 1%
    patch:
      default:
        target: 80%
        threshold: 1%

comment:
  layout: "header, diff, flags, files, footer"
  behavior: default
  require_changes: false
```

GitHub Secrets Configuration

Set up the following secrets in your GitHub repository:

Repository Settings > Secrets and Variables > Actions

FTP Configuration

```

FTP_SERVER=ftp.yourserver.com
FTP_USERNAME=your_ftp_username
FTP_PASSWORD=your_ftp_password
FTP_PATH=/path/to/upload/directory

# Codecov Token (optional, for private repos)
CODECOV_TOKEN=your_codecov_token

# Personal Access Token for releases
GITHUB_TOKEN=automatically_provided_by_github

```

GitLab Variables Configuration

Set up the following variables in your GitLab project:

```

# Project Settings > CI/CD > Variables

# FTP Configuration
FTP_SERVER=ftp.yourserver.com (Type: Variable, Protected: Yes)
FTP_USERNAME=your_ftp_username (Type: Variable, Protected: Yes)
FTP_PASSWORD=your_ftp_password (Type: Variable, Protected: Yes, Masked: Yes)
FTP_PATH=/path/to/upload/directory (Type: Variable, Protected: Yes)

# Codecov Token (if using private repository)
CODECOV_TOKEN=your_codecov_token (Type: Variable, Protected: Yes, Masked: Yes)

```

Package Development Workflow Scripts

scripts/dev-setup.R

```

#!/usr/bin/env Rscript
# Development setup script

# Install development dependencies
if (!requireNamespace("devtools", quietly = TRUE)) {
  install.packages("devtools")
}

if (!requireNamespace("usethis", quietly = TRUE)) {
  install.packages("usethis")
}

# Install package dependencies
devtools::install_deps(dependencies = TRUE)

# Setup package structure (run once)

```

```
# usethis::use_roxygen_md()
# usethis::use_testthat()
# usethis::use_github_actions_check_standard()
# usethis::use_coverage()
# usethis::use_pkgdown()

# Development workflow
devtools::load_all()
devtools::document()
devtools::test()
devtools::check()

cat("Development environment setup complete!\n")
cat("Use devtools::load_all() to load the package\n")
cat("Use devtools::test() to run tests\n")
cat("Use devtools::check() to check the package\n")
```

scripts/pre-commit-check.R

```
#!/usr/bin/env Rscript
# Pre-commit checks script

cat("Running pre-commit checks...\n")

# Document the package
cat("1. Documenting package...\n")
devtools::document()

# Run tests
cat("2. Running tests...\n")
test_results <- devtools::test()

if (length(test_results$failed) > 0) {
  cat("❌ Tests failed!\n")
  quit(status = 1)
}

# Run R CMD check
cat("3. Running R CMD check...\n")
check_results <- devtools::check(quiet = TRUE)

if (length(check_results$errors) > 0 || length(check_results$warnings) > 0) {
  cat("❌ Package check failed!\n")
  print(check_results)
  quit(status = 1)
}
```

```

}

# Check code style (if lintr is available)
if (requireNamespace("lintr", quietly = TRUE)) {
  cat("4. Checking code style...\n")
  lint_results <- lintr::lint_package()
  if (length(lint_results) > 0) {
    cat("▲ Style issues found:\n")
    print(lint_results)
  }
}

cat("☐ All pre-commit checks passed!\n")

```

Performance Monitoring Configuration

.github/workflows/benchmark.yml

```

name: Performance Benchmarks

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  benchmark:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: r-lib/actions/setup-r@v2
        with:
          use-public-rspm: true

      - name: Install system dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y \
            libgsl-dev \
            liblapack-dev \
            libblas-dev \
            libarmadillo-dev \
            libeigen3-dev

```

```
- uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: |
      any::bench
      any::ggplot2

- name: Run benchmarks
  run: |
    R -e "
      library(bench);
      library(YourPackageName);

      # Benchmark key functions
      n <- 10000;
      data <- rnorm(n);

      results <- bench::mark(
        cpp_stats = robust_summary_stats(data),
        r_stats = list(mean = mean(data), sd = sd(data)),
        iterations = 100,
        check = FALSE
      );

      print(results);

      # Save results
      saveRDS(results, 'benchmark_results.rds');

      # Plot results
      ggplot2::ggsave('benchmark_plot.png',
        bench::autoplot(results),
        width = 10, height = 6);
    "

- name: Upload benchmark results
  uses: actions/upload-artifact@v4
  with:
    name: benchmark-results
    path: |
      benchmark_results.rds
      benchmark_plot.png
```

This comprehensive guide provides everything needed to develop, document, test, and deploy R packages with Rcpp integration following CRAN standards and modern CI/CD practices. The

pipeline configurations support automated testing across multiple platforms, code coverage reporting, documentation generation, and automated deployment to FTP servers upon successful builds.

A:1 (Vertex)

[^] *Rcpp Usage Comprehensive Guide*

Here is a comprehensive guide to using Rcpp in R packages, covering everything from coding style to deployment pipelines.

1. C++11 Rcpp Sources Coding Style /v

While CRAN doesn't enforce a strict C++ coding style, it's a best practice to adopt a clear and consistent one. A good approach is to follow the Google C++ Style Guide or a similar widely recognized standard, adapted for Rcpp's specifics.

Good Practices /v

- **Naming:** Use CamelCase for classes and snake_case for variables and functions.
- **Indentation:** Use 2 or 4 spaces for indentation, but be consistent.
- **Comments:** Use `//` for single-line comments and `/* */` for multi-line comments. Document your code clearly.
- **Includes:** Organize includes logically: system headers, then library headers, and finally your project's headers.

Example of Good vs. Bad Practices

Good

```
// Good: Clear, consistent, and commented
#include <Rcpp.h>

// [[Rcpp::export]]
Rcpp::NumericVector calculate_mean(Rcpp::NumericVector x) {
  double total = 0;
  for (int i = 0; i < x.size(); ++i) {
    total += x[i];
  }
  return Rcpp::NumericVector::create(total / x.size());
}
```

Bad

```
// Bad: Inconsistent formatting, poor naming, and no comments
#include <Rcpp.h>
```

```
using namespace Rcpp;
// [[Rcpp::export]]
NumericVector Mean(NumericVector Y){
  double T=0;for(int i=0;i<Y.size();++i){T+=Y[i];}
  return NumericVector::create(T/Y.size());}
```

2. Rcpp Namespace Utility and Best Practices /v

The Rcpp namespace provides a suite of classes and functions that facilitate the interaction between R and C++.

- **I/O:** Use `Rcpp::Rcout` for standard output and `Rcpp::Rcerr` for error output. Avoid using `std::cout` and `std::cerr` as they can interfere with R's I/O streams.
- **Memory Management:** Rcpp automatically handles memory management for objects passed between R and C++. For manual memory management, use `R_alloc` for temporary allocations that are automatically freed.
- **Exception Handling:** Use `Rcpp::stop()` to throw an exception that can be caught by R. This ensures a graceful exit and proper error handling within the R environment.

3. RcppArmadillo and RcppEigen /v

`RcppArmadillo` and `RcppEigen` are packages that integrate the Armadillo and Eigen C++ linear algebra libraries with R, respectively. They offer a more intuitive and efficient way to perform complex matrix operations.

- **RcppArmadillo:** Known for its user-friendly API, making it easy to translate R code into C++.
- **RcppEigen:** Often provides better performance, especially for more complex linear algebra tasks.

RcppArmadillo Example

```
#include <RcppArmadillo.h>
// [[Rcpp::depends(RcppArmadillo)]]

// [[Rcpp::export]]
arma::mat matrix_multiply(const arma::mat& A, const arma::mat& B) {
  return A * B;
}
```

RcppEigen Example

```
#include <RcppEigen.h>
// [[Rcpp::depends(RcppEigen)]]

// [[Rcpp::export]]
Eigen::MatrixXd matrix_multiply_eigen(const Eigen::MatrixXd& A, const Eigen::MatrixXd& B) {
```

```
return A * B;
}
```

4. Integration with Tidyverse Packages (e.g., dplyr) /v

Integrating Rcpp with tidyverse packages like dplyr allows you to create highly efficient data manipulation pipelines. You can write performance-critical operations in C++ and seamlessly call them from within a dplyr workflow.

Example:

```
#include <Rcpp.h>

// [[Rcpp::export]]
Rcpp::NumericVector custom_summarize(Rcpp::DataFrame df) {
  Rcpp::NumericVector x = df["x"];
  double sum_val = 0;
  for (int i = 0; i < x.size(); ++i) {
    sum_val += x[i];
  }
  return Rcpp::NumericVector::create(sum_val);
}
```

R side:

```
library(dplyr)
my_data <- tibble(x = 1:10, group = rep(c("A", "B"), each = 5))
my_data %>%
  group_by(group) %>%
  summarise(custom_sum = custom_summarize(cur_data()))
```

5. Integration with Standard OS Libraries (e.g., GNU Scientific Library) /v

Rcpp can be integrated with external libraries like the GNU Scientific Library (GSL) to leverage a vast collection of numerical routines. The RcppGSL package simplifies this by providing wrapper functions and handling the necessary linking.

To link against GSL, your package's DESCRIPTION file must be correctly configured:

- **SystemRequirements:** Specify the external library needed (e.g., GNU GSL).
- **LinkingTo:** Add RcppGSL to this field.

6. Best Practices for Roxygen2 Comments and Function Documentation /v

Roxygen2 is the standard for documenting R packages, and it works seamlessly with Rcpp.

Roxygen2 Best Practices

- Use `//'` for Roxygen comments in C++ files.
- Document every function with `@param` for each parameter, `@return` for the return value, and `@export` to make the function available in R.
- Provide a clear and concise description of the function's purpose.

Example:

```

//' Calculate the sum of a numeric vector.
//'
//' @param x A numeric vector.
//' @return The sum of the elements in the vector.
//' @export
// [[Rcpp::export]]
double sum_vector(Rcpp::NumericVector x) {
  return Rcpp::sum(x);
}

```

7. R and C++ Function Calls in Rcpp Source /v

You can call R functions from C++ using `Rcpp::Function`. This is useful for accessing R's statistical functions or for callbacks. Similarly, you can call other C++ functions within the same package, which helps in modularizing your code.

Example:

```

#include <Rcpp.h>

// [[Rcpp::export]]
double call_r_mean(Rcpp::NumericVector x) {
  Rcpp::Function r_mean("mean");
  return Rcpp::as<double>(r_mean(x));
}

```

8. Rebuild Operations /v

To rebuild your Rcpp functions, you can use several methods:

- **Rcpp::sourceCpp():** Quickly compile and load a single C++ file for testing and development.
- **R CMD build and R CMD check:** Standard R commands that will compile your C++ code as part of the package building and checking process.
- In RStudio, the "Build and Reload" button (Ctrl+Shift+B) automates this process.

9. Code Unit Testing Support with 'testthat' Package /v

The testthat package is the go-to for unit testing in R, and it can be used to test your Rcpp functions. You can write tests that call your exported C++ functions and compare the results against expected values.

Example:

```
# In tests/testthat/test-my_rcpp_functions.R
library(testthat)
library(yourPackageName)

test_that("sum_vector calculates the correct sum", {
  expect_equal(sum_vector(c(1, 2, 3)), 6)
})
```

10. Code Examples

Full Skeleton of a C++ Source /v

```
// [[Rcpp::plugins(cpp11)]]

#include <RcppArmadillo.h>
#include <RcppEigen.h>
#include <Rcpp.h>
#include <gsl/gsl_matrix.h>
#include <vector>

// [[Rcpp::depends(RcppArmadillo, RcppEigen, RcppGSL)]]

//' @title A sample Rcpp function
//' @description This is a sample function that demonstrates the use of Rcpp.
//' @param x A numeric vector.
//' @return The input vector multiplied by 2.
//' @export
// [[Rcpp::export]]
Rcpp::NumericVector my_cpp_function(Rcpp::NumericVector x) {
  return x * 2;
}
```

Roxygen2 Documentation and DESCRIPTION File Requisites /v

For a package using Rcpp, RcppEigen, and GSL, your DESCRIPTION file should include:

```
Package: myPackage
Type: Package
Title: My Awesome Package
```

```
Version: 0.1.0
Imports: Rcpp
LinkingTo: Rcpp, RcppEigen, RcppGSL
SystemRequirements: GNU GSL
```

Your Roxygen2 documentation should clearly document each function, its parameters, and what it returns, as shown in the examples above.

GitHub and GitLab Packaging Action Pipeline /v

GitHub Actions (.github/workflows/R-CMD-check.yaml)

```
on: [push, pull_request]

name: R-CMD-check

jobs:
  R-CMD-check:
    runs-on: ${{ matrix.config.os }}

    strategy:
      matrix:
        config:
          - {os: macOS-latest, r: 'release'}
          - {os: windows-latest, r: 'release'}
          - {os: ubuntu-latest, r: 'devel', http-user-agent: 'release'}
          - {os: ubuntu-latest, r: 'release'}
          - {os: ubuntu-latest, r: 'oldrel-1'}

    env:
      GITHUB_PAT: ${{ secrets.GITHUB_TOKEN }}

    steps:
      - uses: actions/checkout@v3

      - uses: r-lib/actions/setup-r@v2
        with:
          r-version: ${{ matrix.config.r }}
          http-user-agent: ${{ matrix.config.http-user-agent }}
          use-public-rspm: true

      - name: Install GSL
        run: |
          sudo apt-get update
          sudo apt-get install -y libgsl-dev
```

```

- uses: r-lib/actions/setup-r-dependencies@v2
with:
extra-packages: any::rcmdcheck
needs: check

- uses: r-lib/actions/check-r-package@v2

- name: Upload binary package
  if: success()
  uses: actions/upload-artifact@v3
  with:
    name: package-binary
    path: '*.Rcheck/

```

GitLab CI (.gitlab-ci.yml)

```
image: rocker/tidyverse:latest
```

```
stages:
```

```

- build
- test
- deploy

```

```
before_script:
```

```

- apt-get update && apt-get install -y libgsf-dev
- R -e "install.packages(c('rcpp', 'devtools', 'testthat'))"

```

```
build:
```

```

stage: build
script:
- R CMD build .
artifacts:
paths:
- "*.tar.gz"

```

```
test:
```

```

stage: test
script:
- R CMD check --as-cran *.tar.gz

```

```
deploy_ftp:
```

```

stage: deploy
script:
- apt-get install -y lftp
- lftp -e "set ftp:ssl-allow no; put -O . $(ls *.tar.gz); bye" -u $FTP_USER,$FTP_PASSWORD $FTP_

```

```
only:  
- main
```

A:1 (Gemini)

^ *Rcpp Programming Guide*

Here is a comprehensive guide on Rcpp usage for R packages, adhering to CRAN requirements and best practices.

1. C++11 Rcpp Sources Coding Style /g

Adhering to a consistent coding style is crucial for readability and maintainability, especially in a collaborative environment. While C++ has several established style guides (like Google's or Stroustrup's), the key for R packages is consistency and clarity.

Recommendations:

- **Naming Conventions:**
 - Use `CamelCase` for function names and class names (e.g., `myFunction`, `MyClass`).
 - Use `snake_case` for variable names (e.g., `my_variable`).
 - Use `UPPER_SNAKE_CASE` for constants
- **Indentation and Braces:**
 - Use 2 or 4 spaces for indentation, but be consistent.
 - Place opening braces on the same line as the statement (`{`) and closing braces on a new line (`}`).
- **Comments:**
 - Use `//` for single-line comments and `/* ... */` for multi-line comments.
 - Comment complex logic, but avoid commenting on obvious code.
- **Header Guards:**
 - Use `#pragma once` or traditional header guards (`#ifndef MY_HEADER_H ... #endif`) to prevent multiple inclusions.

Good Practice Example:

```
// [[Rcpp::plugins(cpp11)]]
#include <Rcpp.h>

//' A function to add two numbers.
//'
//' @param x An integer.
//' @param y An integer.
//' @return The sum of x and y.
//' @export
// [[Rcpp::export]]
```

```
int add_numbers(int x, int y) {
    int sum_result = x + y;
    return sum_result;
}
```

Bad Practice Example:

```
#include <Rcpp.h>
//No C++11 plugin enabled

// No Roxygen documentation
int add(int A, int B){ // Inconsistent bracing and capitalization
double res = A+B; // Unnecessary type conversion
return res;} // Crowded return statement
```

2. Rcpp Namespace Utility and Best Practices /g

The Rcpp namespace is the heart of Rcpp, providing the bridge between R and C++.

- **Utility:** It contains classes that map R objects to C++ objects (e.g., NumericVector, DataFrame, Function) and functions that allow interaction with the R environment.
- **Basic I/O:** For printing to the R console, use `Rcpp::Rcout << "message";` instead of `std::cout`. For warnings and errors, use `Rcpp::warning("message");` and `Rcpp::stop("message");`.
- **Memory Management:** Rcpp handles memory management automatically for its objects through RAII (Resource Acquisition Is Initialization). When an Rcpp object goes out of scope, the memory for the corresponding R object is released. You generally do not need to manually allocate or deallocate memory for Rcpp objects.
- **Exception Handling:** C++ exceptions are automatically caught by Rcpp and converted into R errors. This means you can use try-catch blocks in your C++ code, and any uncaught exceptions will be properly handled by R.

Best Practice:

```
#include <Rcpp.h>

// [[Rcpp::export]]
void process_data(Rcpp::NumericVector data) {
    try {
        if (data.size() == 0) {
            Rcpp::stop("Input data cannot be empty.");
        }
        // ... processing logic ...
        Rcpp::Rcout << "Processing complete." << std::endl;
    } catch (std::exception &ex) {
        Rcpp::warning("An exception occurred: %s", ex.what());
    }
}
```

3. RcppArmadillo and RcppEigen /g

These packages provide high-performance linear algebra libraries for C++, seamlessly integrated with R.

- **RcppArmadillo:**
 - **Purpose:** Provides the Armadillo C++ library, which offers a high-level syntax for linear algebra, similar to MATLAB.
 - **Benefits:** User-friendly syntax, good performance, and extensive documentation. It's often easier to learn for those new to C++ linear algebra.
- **RcppEigen:**
 - **Purpose:** Provides the Eigen C++ library, another powerful template library for linear algebra.
 - **Benefits:** Excellent performance, especially for small to medium-sized matrices, and a flexible API. It is widely used in high-performance computing.

RcppArmadillo Example:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::export]]
arma::mat invert_matrix_arma(const arma::mat& A) {
  return arma::inv(A);
}
```

RcppEigen Example:

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>

// [[Rcpp::export]]
Eigen::MatrixXd invert_matrix_eigen(const Eigen::MatrixXd& A) {
  return A.inverse();
}
```

4. Integration with Tidyverse Packages (e.g., dplyr) /g

You can integrate Rcpp with tidyverse packages by creating functions in C++ that can be called within a dplyr pipeline. This is often done for performance-critical operations.

Example:

Let's create a C++ function to compute a rolling average and use it with dplyr.

C++ Code (src/rolling_average.cpp)

```
// [[Rcpp::plugins(cpp11)]]
#include <Rcpp.h>

// [[Rcpp::export]]
Rcpp::NumericVector rolling_average(Rcpp::NumericVector x, int n) {
  int len = x.size();
  Rcpp::NumericVector out(len);

  for(int i = 0; i < len; ++i) {
    double sum = 0;
    int count = 0;
    for(int j = std::max(0, i - n + 1); j ≤ i; ++j) {
      sum += x[j];
      count++;
    }
    out[i] = sum / count;
  }
  return out;
}
```

R Code

```
library(dplyr)
library(Rcpp)

# Assume the package is loaded, making rolling_average available
# sourceCpp("src/rolling_average.cpp") # for interactive testing

my_data <- tibble(
  group = rep(c("a", "b"), each = 10),
  value = rnorm(20)
)

my_data %>%
  group_by(group) %>%
  mutate(rolled_avg = rolling_average(value, n = 3))
```

5. Integration with Standard OS Libraries (e.g., gsl - GNU Scientific Library)

To use external libraries like GSL, you need to configure your package to link against them.

Package Requirements:

1. **DESCRIPTION File:** You must declare the dependency.

```
LinkingTo: Rcpp, RcppArmadillo, RcppEigen
SystemRequirements: GSL (≥ 2.0)
```

2. **src/Makevars and src/Makevars.win:** You need to tell the compiler where to find the GSL headers and libraries.

src/Makevars.win (for Windows): This is more complex as it often requires bundling the library or providing instructions for the user to install it. For R versions 4.2 and newer, you can often install libraries via rtools.

src/Makevars (for Linux/macOS):

```
PKG_CPPFLAGS = `gsl-config --cflags`
PKG_LIBS = `gsl-config --libs`
```

Example C++ Code:

```
// [[Rcpp::depends(RcppGSL)]] // RcppGSL can simplify things
#include <Rcpp.h>
#include <gsl/gsl_sf_bessel.h>

// [[Rcpp::export]]
double besseL_function_gsl(double x) {
  return gsl_sf_bessel_J0(x);
}
```

6. Best Practices for Roxygen2 Comments and Function Documentation /g

Roxygen2 comments are essential for creating user-friendly documentation.

Best Practices:

- **Header:** Start with a concise title.
- **Description:** Add a more detailed description of what the function does.
- **Parameters (@param):** Describe each parameter, its type, and its purpose.
- **Return Value (@return):** Describe what the function returns.
- **@export Tag:** This makes the function available to R users.
- **@examples Tag:** Provide runnable examples.
- **@seealso Tag:** Link to related functions.
- **@keywords Tag:** (Optional) Add keywords for indexing.

Well-Documented Function Example:

```
/' Calculate the Inverse of a Matrix using Armadillo
/'
/' This function computes the inverse of a square matrix using the
/' RcppArmadillo library. It provides a direct interface to Armadillo's
/' `inv()` function.
/'
```

```

//' @param A A numeric square matrix.
//' @return A numeric matrix that is the inverse of `A`.
//' @seealso \code{\link[base]{solve}}
//' @examples
//' A <- matrix(c(1, 2, 3, 4), nrow = 2)
//' try({
//'   inv_A <- invert_matrix_arma(A)
//'   print(inv_A)
//' })
//' @export
// [[Rcpp::export]]
arma::mat invert_matrix_arma(const arma::mat& A) {
  if (A.n_rows != A.n_cols) {
    Rcpp::stop("Input matrix must be square.");
  }
  return arma::inv(A);
}

```

7. R and C++ Function Calls in Rcpp Source /g

You can call other R and C++ functions from your Rcpp code.

Calling another C++ function in the same package

Simply define the function in a header file and include it.

src/my_helpers.h

```

#ifndef MY_HELPERS_H
#define MY_HELPERS_H

double internal_calculation(double x);

#endif

```

src/my_helpers.cpp

```

#include "my_helpers.h"

double internal_calculation(double x) {
  return x * x;
}

```

src/main.cpp

```
#include <Rcpp.h>
#include "my_helpers.h"

// [[Rcpp::export]]
double public_function(double val) {
  return internal_calculation(val);
}
```

Calling an R function from Rcpp

1. **Get the function:** Use `Rcpp::Function` to get the R function.
2. **Call it:** Execute it like a C++ function.

```
#include <Rcpp.h>

// [[Rcpp::export]]
Rcpp::NumericVector call_r_function(Rcpp::NumericVector x) {
  Rcpp::Function r_mean("mean");
  return r_mean(x);
}
```

Calling a function from another R package

Specify the package namespace when getting the function.

```
#include <Rcpp.h>

// [[Rcpp::export]]
Rcpp::RObject call_dplyr_mutate() {
  Rcpp::Environment dplyr_ns = Rcpp::Environment::namespace_env("dplyr");
  Rcpp::Function mutate = dplyr_ns["mutate"];

  // Example usage (conceptual)
  // ... create a data frame and arguments ...
  // return mutate( ... );
  return R_NilValue; // Placeholder
}
```

8. Rebuild Operations /g

- **Rcpp::sourceCpp():** For interactive development, this function compiles and sources a single C++ file.
- **devtools::load_all():** During package development, this will automatically recompile any changed C++ files.

- **R CMD INSTALL** or `devtools::install()`: This builds and installs the entire package, compiling all C++ source code.
- **R CMD check** or `devtools::check()`: This also recompiles the C++ code as part of the checking process.

The most common and recommended workflow during development is to use `devtools::load_all()` to incrementally recompile changes.

9. Code Unit Testing Support with 'testthat' Package /g

testthat is the standard for unit testing in R, and it works seamlessly with Rcpp.

1. **Setup:** Use `usethis::use_testthat()`. This creates the `tests/testthat/` directory and a `tests/testthat.R` file.
2. **Create Test Files:** Add files like `tests/testthat/test-my_rcpp_functions.R`.
3. **Write Tests:** Use `test_that()` blocks to organize tests. Inside these, use `expect_*()` functions to check your C++ function's behavior.

Example Test File (`tests/testthat/test-add_numbers.R`):

```
library(testthat)
# Assuming the package is named 'myPackage'
library(myPackage)

context("Test add_numbers C++ function")

test_that("add_numbers works with positive integers", {
  expect_equal(add_numbers(2, 3), 5)
})

test_that("add_numbers works with zero", {
  expect_equal(add_numbers(5, 0), 5)
})

test_that("add_numbers works with negative numbers", {
  expect_equal(add_numbers(-1, -5), -6)
})

test_that("add_numbers handles data types correctly", {
  expect_type(add_numbers(1, 2), "integer")
})
```

Full Skeleton of a C++ Source /g

Here is a full skeleton of a C++ source file (`src/my_cpp_functions.cpp`) with best practices.

```

// Enable C++11
// [[Rcpp::plugins(cpp11)]]

// Dependencies
// [[Rcpp::depends(RcppArmadillo, RcppEigen, RcppGSL)]]

// Standard Library Headers
#include <iostream>
#include <vector>

// Rcpp Headers
#include <Rcpp.h>
#include <RcppArmadillo.h>
#include <RcppEigen.h>

// GSL Headers (assuming RcppGSL is used or linked properly)
#include <gsl/gsl_sf_bessel.h>

// Internal Helper Functions (not exported to R)
namespace {
    double internal_helper_function(double x) {
        return x + 1.0;
    }
}

//' @title A C++ function using Armadillo
//' @description This function demonstrates basic Armadillo usage.
//' @param M A numeric matrix.
//' @return The transpose of the matrix.
//' @export
// [[Rcpp::export]]
arma::mat my_arma_function(arma::mat M) {
    return M.t();
}

//' @title A C++ function using Eigen
//' @description This function demonstrates basic Eigen usage.
//' @param A A numeric matrix.
//' @return The inverse of the matrix.
//' @export
// [[Rcpp::export]]
Eigen::MatrixXd my_eigen_function(Eigen::MatrixXd A) {
    return A.inverse();
}

```

```

//' @title A C++ function using GSL
//' @description This function calls a GSL special function.
//' @param x A numeric value.
//' @return The J0 Bessel function of x.
//' @export
// [[Rcpp::export]]
double my_gsl_function(double x) {
  return gsl_sf_bessel_J0(x);
}

//' @title A C++ function calling an R function
//' @description Demonstrates how to call an R function from C++.
//' @param vec A numeric vector.
//' @return The standard deviation calculated by R's `sd()` function.
//' @export
// [[Rcpp::export]]
double call_r_sd(Rcpp::NumericVector vec) {
  Rcpp::Function r_sd("sd");
  Rcpp::NumericVector result = r_sd(vec);
  return result[0];
}

```

Roxygen2 Documentation and DESCRIPTION File Requisites /g

Roxygen2 Documentation:

As shown in the skeleton above, every exported C++ function should have a Roxygen2 block directly above its signature. The C++ code is parsed by `Rcpp::compileAttributes()`, which generates the necessary R wrapper code and .Rd documentation files.

DESCRIPTION File:

```

Package: MyPackageName
Type: Package
Title: An Example Rcpp Package
Version: 0.1.0
Author: Your Name
Maintainer: Your Name <your.email@example.com>
Description: A package to demonstrate Rcpp, RcppArmadillo, RcppEigen, and GSL integration.
License: GPL-3
Encoding: UTF-8
LazyData: true
RoxygenNote: 7.2.3
Imports:
  Rcpp (≥ 1.0.0)
LinkingTo:

```

```
Rcpp,
RcppArmadillo,
RcppEigen,
RcppGSL
```

SystemRequirements: `GSL (≥ 2.0)`

Depends:

```
R (≥ 3.5.0)
```

Key Points:

- **Imports:** You must import Rcpp.
- **LinkingTo:** This is crucial. It tells R where to find the header files for the other C++ libraries at compile time.
- **SystemRequirements:** This informs users (and automated systems) about external non-R dependencies. It doesn't install them but provides a human-readable note.

GitHub and GitLab Packaging Action Pipeline /g

Here are example CI/CD pipelines for GitHub Actions and GitLab CI.

GitHub Actions

Create the file `.github/workflows/R-CMD-check.yaml`:

```
# Workflow for building and checking an R package
#
# This workflow runs the standard R checks on a package but also shows
# how to upload the built package to a hypothetical FTP server.
```

```
name: R-CMD-check
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
      - master
```

```
  pull_request:
```

```
    branches:
```

```
      - main
      - master
```

```
jobs:
```

```
  R-CMD-check:
```

```
    runs-on: ${{ matrix.config.os }}
```

```
    name: ${{ matrix.config.os }} (${{ matrix.config.r }}
```

```
strategy:
  fail-fast: false
  matrix:
    config:
      - {os: macOS-latest, r: 'release'}
      - {os: windows-latest, r: 'release'}
      - {os: ubuntu-latest, r: 'devel', http-user-agent: 'release'}
      - {os: ubuntu-latest, r: 'release'}
      - {os: ubuntu-latest, r: 'oldrel-1'}

env:
  GITHUB_PAT: ${ secrets.GITHUB_TOKEN }
  R_KEEP_PKG_SOURCE: yes
  FTP_USER: ${ secrets.FTP_USER }
  FTP_PASSWORD: ${ secrets.FTP_PASSWORD }

steps:
  - uses: actions/checkout@v3

  - uses: r-lib/actions/setup-pandoc@v2

  - uses: r-lib/actions/setup-r@v2
    with:
      r-version: ${ matrix.config.r }
      http-user-agent: ${ matrix.config.http-user-agent }
      use-public-rspm: true

  - name: Install GSL (Linux)
    if: runner.os == 'Linux'
    run: sudo apt-get update && sudo apt-get install -y libgsl-dev

  - name: Install GSL (macOS)
    if: runner.os == 'macOS'
    run: brew install gsl

  - name: Install GSL (Windows)
    if: runner.os == 'Windows'
    run: pacman -S --noconfirm mingw-w64-x86_64-gsl

  - uses: r-lib/actions/setup-r-dependencies@v2
    with:
      extra-packages: any::rcmdcheck
      needs: check
```

```

- name: Check package
  id: check
  uses: r-lib/actions/check-r-package@v2
  with:
    upload-snapshots: true

- name: Build binary package
  if: success()
  run: |
    install.packages("devtools")
    devtools::build(binary = TRUE)
  shell: Rscript {0}

- name: Upload binary package to FTP
  if: success()
  uses: SamKirkland/FTP-Deploy-Action@4.3.3
  with:
    server: ftp.example.com
    username: ${ env.FTP_USER }
    password: ${ env.FTP_PASSWORD }
    local-dir: ./
    # Find the built binary file (extension varies by OS)
    # This is a simplified approach; a more robust script might be needed.
    state-name: ${ steps.check.outputs.package-tarball }

```

Notes for GitHub Actions:

- You need to add FTP_USER and FTP_PASSWORD to your repository's secrets under Settings > Secrets and variables > Actions.
- The FTP-Deploy-Action is a third-party action.

GitLab CI/CD

Create the file `.gitlab-ci.yml` in the root of your repository.

```

# .gitlab-ci.yml
# This pipeline checks an R package and uploads a binary to FTP.

```

```

image: rocker/verse:latest

```

variables:

```

# FTP_USER and FTP_PASSWORD should be configured in GitLab's
# CI/CD settings → variables (masked and protected)
FTP_SERVER: ftp.example.com

```

stages:

```

- test
- build
- deploy

before_script:
- apt-get update && apt-get install -y libgsf-dev lftp
- R -e 'install.packages(c("rcmdcheck", "devtools", "Rcpp", "RcppArmadillo", "RcppEigen", "Rcp

test:
  stage: test
  script:
    - rcmdcheck::rcmdcheck(args = "--no-manual", error_on = "warning")

build_binary:
  stage: build
  script:
    - R -e 'devtools::build(binary = TRUE)'
  artifacts:
    paths:
      - "*.zip" # Or *.tgz depending on the OS
    expire_in: 1 week

deploy_ftp:
  stage: deploy
  script:
    # Find the binary package file created in the build stage
    - BINARY_FILE=$(ls *.zip 2>/dev/null || ls *.tgz 2>/dev/null)
    - |
      if [ -z "$BINARY_FILE" ]; then
        echo "Binary package file not found!"
        exit 1
      fi
    - echo "Uploading $BINARY_FILE to $FTP_SERVER"
    - lftp -u "$FTP_USER,$FTP_PASSWORD" $FTP_SERVER -e "set ftp:ssl-allow no; put $BINARY_FILE;"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"' # Only deploy from the main branch
  needs:
    - job: build_binary
      artifacts: true

```

Notes for GitLab CI:

- Add FTP_USER and FTP_PASSWORD as protected and masked variables in your project's Settings > CI/CD > Variables.
- This example uses the rocker/verse Docker image, which comes with many R packages pre-installed. You may need to adjust the before_script depending on your exact needs.

A:1 (ChatGPT)

^ *Rcpp usage guide*

Here is a comprehensive guide to **Rcpp usage in R packages**, following CRAN compliance and best practices.

□ Guide to Rcpp Usage in R Packages

1. C++11 Rcpp Sources Coding Style

▮ Recommended Style

- Follow Google C++ Style Guide adapted for Rcpp.
- Use camelCase for function names and snake_case for variables.
- Brace on the same line: `if (cond) {`
- Proper indentation (2 or 4 spaces).
- Prefer `auto` when type is obvious.
- Avoid using `namespace std;` or `Rcpp;` in headers.

▮ Good Practice

```
// [[Rcpp::export]]
int sumVec(Rcpp::IntegerVector x) {
    int total = 0;
    for (int i = 0; i < x.size(); ++i) {
        total += x[i];
    }
    return total;
}
```

▮ Bad Practice

```
// [[Rcpp::export]]
int sumvec(Rcpp::IntegerVector x){
int total=0;
for(int i=0;i<x.size();i++){
total+=x[i];}
return total;}
```

2. Rcpp Namespace Utility and Best Practices

Utility

- `Rcpp::` wraps native R types for seamless C++ use.
- Provides data structures (e.g., `NumericVector`, `List`) and helper macros like `RCPP_MODULE`.

Best Practices

- Use `Rcpp::Named()` for named list output.
- Always handle exceptions with `try-catch`.
- Avoid raw pointers; prefer STL containers or Rcpp equivalents.

Example

```
// [[Rcpp::export]]
Rcpp::List stats(Rcpp::NumericVector x) {
  double mean = Rcpp::mean(x);
  double sd = Rcpp::sd(x);
  return Rcpp::List::create(Rcpp::Named("mean") = mean,
                           Rcpp::Named("sd") = sd);
}
```

3. RcppArmadillo and RcppEigen

Purpose

- **RcppArmadillo**: Linear algebra using Armadillo (efficient C++ matrix library).
- **RcppEigen**: Interfaces with Eigen, another high-performance matrix library.

RcppArmadillo Example

```
#include <RcppArmadillo.h>
// [[Rcpp::depends(RcppArmadillo)]]
// [[Rcpp::export]]
arma::mat matMult(arma::mat A, arma::mat B) {
  return A * B;
}
```

RcppEigen Example

```
#include <RcppEigen.h>
// [[Rcpp::depends(RcppEigen)]]
```

```
// [[Rcpp::export]]
Eigen::MatrixXd matMultEigen(Eigen::MatrixXd A, Eigen::MatrixXd B) {
  return A * B;
}
```

4. Integration with Tidyverse Packages (e.g., dplyr)

Rcpp functions can interoperate with tidyverse by using `dplyr::mutate()` with `.data` as input.

Example

```
library(Rcpp)
library(dplyr)

cppFunction('
NumericVector square(NumericVector x) {
  return x * x;
}
')

df <- tibble(x = 1:5)
df %>% mutate(x2 = square(x))
```

5. Integration with Standard OS Libraries (e.g., GSL)

Steps

- Install GSL system-wide.
- Add compiler flags via `Makevars` or `Makevars.win`.

DESCRIPTION

LinkingTo: Rcpp, RcppGSL
 SystemRequirements: GNU Scientific Library (GSL)

Example

```
#include <RcppGSL.h>
#include <gsl/gsl_sf_bessel.h>

// [[Rcpp::depends(RcppGSL)]]
// [[Rcpp::export]]
double besselJ0(double x) {
```

```

    return gsl_sf_bessel_J0(x);
}

```

6. Best Practices for Roxygen2 Comments

□ Guidelines

- Always use @export for R-visible functions.
- Use @param, @return, @examples.

□ Example

```

//' Compute sum of vector
//'
//' @param x Numeric vector.
//' @return Sum of elements.
//' @examples
//' cpp_sum(c(1, 2, 3))
// [[Rcpp::export]]
double cpp_sum(Rcpp::NumericVector x) {
    return sum(x);
}

```

7. Calling Other Functions in Rcpp

▮ C++ to C++ (internal function)

```

double helper(double x) {
    return x * x;
}

// [[Rcpp::export]]
double useHelper(double x) {
    return helper(x);
}

```

▮ R call from C++

```

// [[Rcpp::export]]
SEXP call_r_fun(SEXP x) {
    Rcpp::Function f("log");
    return f(x);
}

```

8. Rebuild Operations

□ Options:

- Rebuild with `Rcpp::compileAttributes()`.
- Rebuild package:

R CMD build mypkg

R CMD check mypkg_*.tar.gz

9. Unit Testing with testthat

▮ Setup

- Add testthat to Suggests in DESCRIPTION.
- Enable in tests/testthat.R.

▮ Example Test

```
test_that("cpp_sum works", {
  expect_equal(cpp_sum(c(1, 2, 3)), 6)
})
```

▮ Full Skeleton of a C++ Source File

```
// [[Rcpp::plugins(cpp11)]]
#include <Rcpp.h>
#include <RcppArmadillo.h>
#include <RcppEigen.h>
#include <gsl/gsl_sf_bessel.h>
#include <vector>
#include <string>

// [[Rcpp::depends(RcppArmadillo, RcppEigen, RcppGSL)]]
// [[Rcpp::export]]
Rcpp::List example_function(Rcpp::NumericVector x) {
  arma::vec ax(x.begin(), x.size(), false);
  Eigen::VectorXd ex = Eigen::Map<Eigen::VectorXd>(x.begin(), x.size());
  double bessel_val = gsl_sf_bessel_J0(x[0]);
```

```

return Rcpp::List::create(
  Rcpp::Named("arma_sum") = arma::sum(ax),
  Rcpp::Named("eigen_sum") = ex.sum(),
  Rcpp::Named("bessel_j0") = bessel_val
);
}

```

■ Roxygen2 + DESCRIPTION Requirements

□ Roxygen Tags

- @useDynLib yourpackage, .registration=TRUE
- @importFrom Rcpp sourceCpp
- @import Rcpp

□ DESCRIPTION

Imports: Rcpp
 LinkingTo: Rcpp, RcppArmadillo, RcppEigen, RcppGSL
 SystemRequirements: GNU Scientific Library (GSL)
 Roxygen: list(markdown = TRUE)
 RoxygenNote: 7.2.3

■ GitHub/GitLab CI Pipeline (R CMD Check + Binary Upload)

■ GitHub Actions .github/workflows/R-CMD-check.yml

```

name: R-CMD-check

on:
  push:
  pull_request:

jobs:
  R-CMD-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
      - uses: r-lib/actions/setup-r-dependencies@v2
      with:
        extra-packages: any::rcmdcheck
      - name: Check

```

```
  run: R CMD check --as-cran .
- name: Build Binary
  run: R CMD INSTALL --build .
- name: Upload to FTP
  if: success()
  env:
    FTP_USER: ${ secrets.FTP_USER }
    FTP_PASS: ${ secrets.FTP_PASS }
  run: |
    curl -T *.tar.gz ftp://ftp.example.com/path/ \
      --user "$FTP_USER:$FTP_PASS"
```

📄 **GitLab .gitlab-ci.yml**

```
check:
  image: rocker/r-ver:4.3.0
  script:
    - R -e 'rcmdcheck::rcmdcheck(args = "--as-cran")'
    - R CMD INSTALL --build .
    - curl -T *.tar.gz ftp://ftp.example.com/path/ --user $FTP_USER:$FTP_PASS
```

Let me know if you need a sample working package layout or help uploading to CRAN.

A:1 (Perplexity)

^ *Rcpp Guide for R Packages*

1. C++11 Rcpp Sources Coding Style /p

Recommended Coding Style /p

- **Indentation:** Use 4 spaces, not tabs.
- **File Names:** Use .cpp for source files.
- **Headers:** Always include RcppArmadillo.h or Rcpp.h or, in alternative, RcppEigen.h , and only include other headers as needed.
- **Namespaces:** Prefer Rcpp:: prefix instead of using namespace Rcpp; to avoid namespace pollution.
- **Bracing and Alignment:** Place braces on the same line. Align control statements and use clear, readable formatting.
- **Variable Naming:** Use snake_case or camelCase consistently.
- **Auto:** Use auto wisely to improve clarity but not to hide types unnecessarily (a C++11 feature).
- **Return Types:** Use explicit return types for functions.
- **Initialization:** Use brace {} initialization where possible (C++11 feature).
- **Comments:** Prefer block (/* ... */) or Doxygen-style comments for documentation.

Example: Good Practice

```
#include

// [[Rcpp::export]]
int sum_cpp(const std::vector& x) {
    int total = 0;
    for (const auto& value : x) {
        total += value;
    }
    return total;
}
```

Example: Bad Practice

```
#include
using namespace Rcpp;
int sum_cpp(vector x){
int total=0;for(int i=0;i
```

```
// [[Rcpp::depends(RcppArmadillo)]]
// [[Rcpp::export]]
arma::mat mat_mult(const arma::mat& A, const arma::mat& B) {
  return A * B;
}
```

RcppEigen /p

- **Purpose:** Interface to Eigen library, providing big and small dense/sparse matrix computations¹².
- **Benefits:** Fast, powerful, elegant syntax, and advanced solvers.

Example

```
#include
// [[Rcpp::depends(RcppEigen)]]
// [[Rcpp::export]]
Eigen::VectorXd get_eigenvalues(Eigen::MatrixXd M) {
  Eigen::SelfAdjointEigenSolver es(M);
  return es.eigenvalues();
}
```

4. Integration with Tidyverse Packages (e.g., dplyr) /p

- **Usage:** Use `Rcpp::DataFrame`, `NumericVector` and connect with dplyr-processed data³.
- **Dependency:** Add `// [[Rcpp::depends(dplyr)]]` in the source and corresponding entries in the DESCRIPTION.

Example

```
#include
// [[Rcpp::depends(dplyr)]]
// [[Rcpp::export]]
Rcpp::DataFrame add_one(Rcpp::DataFrame df) {
  Rcpp::NumericVector x = df["x"];
  x = x + 1;
  df["x"] = x;
  return df;
}
```

Used in R after dplyr mutate/select, or inside a dplyr pipeline using `do()`.

¹[RcppEigen](#)

²[an-rcppeigen-example](#)

³[ordering-dataframe-using-rcpp-dplyr](#)

5. Integration with Standard OS Libraries (e.g., GSL) /p

- **Integration:** Use “ etc. in your C++ files. For smoother R interfacing, use RcppGSL⁴⁵.
- **DESCRIPTION requirements:**
 - LinkingTo: Rcpp, RcppGSL
 - SystemRequirements: GNU GSL
- **Linkage:** Configure src/Makevars using `gsl-config --cflags` and `--libs` or use `configure.ac` for portability⁶⁷.

Example:

```
#include
#include
// [[Rcpp::export]]
double gsl_bessel0(double x) {
    return gsl_sf_bessel_J0(x);
}
```

6. Best Practices for Roxygen2 Comments and Function Documentation /p

- **Place above function, before `// [[Rcpp::export]]`.**
- **Include:** Title, description, parameters, return, examples, and `@export`.
- **Document all argument types and units.**

Example:

```
//' Sum elements of a vector
//'
//' Computes the sum of the supplied numeric vector.
//' @param x A numeric vector.
//' @return Numeric scalar. The sum of \code{x}.
//' @examples
//' sum_cpp(c(1, 2, 3))
//' @export
// [[Rcpp::export]]
double sum_cpp(Rcpp::NumericVector x) {
    return std::accumulate(x.begin(), x.end(), 0.0);
}
```

7. R and C++ Function Calls in Rcpp Source /p

- **C++ Function From Package:** Include in same or another .cpp source. Declare its prototype.

⁴[CRAN:RcppGSL](#)

⁵[RcppGSL](#)

⁶[RcppGSL-intro.pdf](#)

⁷[linking-rcpp-to-interp2d-gsl-type-library](#)

- **External R Function:** Use `Rcpp::Function`.

Example: *Calling an R function**

```
// [[Rcpp::export]]
Rcpp::NumericVector call_mean(Rcpp::NumericVector x) {
  Rcpp::Function mean("mean");
  return mean(x);
}
```

Example: C++ function calling another C++ function

```
double helper(double x) { return x * 2; }

// [[Rcpp::export]]
double use_helper(double x) { return helper(x); }
```

8. Rebuild Operations /p

- **Re-compile in R:** `devtools::load_all()` or `Rcpp::compileAttributes()`, or touch source and run.
- **Command line:** R CMD build ., R CMD check --as-cran .
- **RStudio:** Use "Build & Reload".

9. Code Unit Testing Support with 'testthat'

- Put test files in `tests/testthat/` directory.
- Call exported Rcpp functions in test files as for R functions⁸.

Example

```
test_that("sum_cpp works for a numeric vector", {
  expect_equal(sum_cpp(c(1,2,3,4)), 10)
  expect_error(sum_cpp(NULL))
})
```

1. Full Skeleton of a C++ Source

```
// Enable C++11
// [[Rcpp::plugins(cpp11)]]
// [[Rcpp::depends(RcppArmadillo)]]
// [[Rcpp::depends(RcppEigen)]]
// [[Rcpp::depends(RcppGSL)]]
// [[Rcpp::depends(dplyr)]]
```

⁸[testing-rcpp-packages](#)

```

#include
#include
#include
#include
#include
#include
#include

// [[Rcpp::export]]
double example_function(Rcpp::NumericVector x) {
  // Use Rcpp and STL idioms
  return std::accumulate(x.begin(), x.end(), 0.0);
}

```

2. Roxygen2 Documentation and DESCRIPTION File Requisites

Roxygen2:

- Add `@useDynLib` , `.registration = TRUE` in your main package R file.
- Add `@importFrom Rcpp sourceCpp` and document all parameters and return values⁹¹⁰.

DESCRIPTION File:

```

Imports: Rcpp (≥ 1.0.0), dplyr
LinkingTo: Rcpp, RcppArmadillo, RcppEigen, RcppGSL
SystemRequirements: GNU GSL
Suggests: testthat

```

3. GitHub and GitLab Packaging Action Pipeline

Here is a minimal **GitHub Actions** workflow and a conceptually similar **GitLab CI** pipeline. Both run R CMD check and, if successful, upload the built package to a remote FTP server using provided environment variables.

GitHub Actions: `.github/workflows/R-CMD-check.yaml`

```

name: R-CMD-check

on: [push, pull_request]

jobs:
  R-CMD-check:
    runs-on: ubuntu-latest

```

⁹[documenting-rcpp-packages](#)

¹⁰[a-minimal-rcpp-and-roxygen2-package](#)

```

steps:
- uses: actions/checkout@v4
- uses: r-lib/actions/setup-r@v2
- name: Install dependencies
  run: |
    install.packages(c('devtools', 'Rcpp', 'RcppArmadillo', 'RcppEigen', 'RcppGSL', 'dplyr',
- name: Check
  run: R CMD check --as-cran .
- name: Build
  run: R CMD build .
- name: Upload to FTP
  if: success()
  run: |
    curl -T *.tar.gz -u "${{ secrets.FTP_USER }}:${{ secrets.FTP_PASSWORD }}" "ftp://{{ secrets.FTP_HOST }}/upload/"

```

GitLab CI: .gitlab-ci.yml

```

stages:
- build
- upload

build_package:
  image: r-base:latest
  script:
    - apt-get update && apt-get install -y libgsl-dev libcurl4-openssl-dev
    - Rscript -e 'install.packages(c("devtools", "Rcpp", "RcppArmadillo", "RcppEigen", "RcppGSL",
    - R CMD check --as-cran .
    - R CMD build .

upload_ftp:
  stage: upload
  script:
    - apt-get update && apt-get install -y curl
    - curl -T *.tar.gz -u "$FTP_USER:$FTP_PASSWORD" "ftp://$FTP_HOST/upload/"
  only:
    - master
  dependencies:
    - build_package
  when: on_success

```

(Set FTP_USER, FTP_PASSWORD, FTP_HOST in repository secrets/variables.)

This guide presents a comprehensive set of up-to-date best practices for using Rcpp and its ecosystem with C++11, armadillo, eigen, dplyr, external libraries like GSL, and robust packaging/testing

workflows that conform to CRAN standards¹¹¹²¹³¹⁴¹⁵¹⁶¹⁷¹⁸¹⁹²⁰²¹²²²³²⁴²⁵²⁶.

¹¹[Rcpp-FAQ.pdf](#)

¹²[CRAN: Rcpp](#)

¹³[rcpparmadillo](#)

¹⁴[RcppEigen](#)

¹⁵[ordering-dataframe-using-rcpp-dplyr](#)

¹⁶[CRAN:RcppGSL](#)

¹⁷[documenting-rcpp-packages](#)

¹⁸[rcpparmadillo-package-in-r](#)

¹⁹[RcppGSL](#)

²⁰[a-minimal-rcpp-and-roxygen2-package](#)

²¹[c11-with-r-and-rcpp-supported-by-cran-policies](#)

²²[rcpp-and-c++11-c++14-c++17](#)

²³[first-steps-with-C++11](#)

²⁴[Rcpp-modules.pdf](#)

²⁵[memory-error-with-rcpparmadillo](#)

²⁶[fastverse](#)

A:1 (DeepSeek)

^ *Comprehensive Guide to Using Rcpp in R Packages*

1. C++11 Rcpp Sources Coding Style

Recommendations:

- **Naming:** Use snake_case for functions/variables, CamelCase for classes
- **Indentation:** 2 spaces (CRAN-compliant)
- **Braces:** Always include {}, even for single-line blocks
- **Headers:** Order includes as: Rcpp, STL, external libraries
- **Comments:** Use // for single-line, /* */ for multi-line

Good Practice:

```
#include <Rcpp.h>
using namespace Rcpp;

// Calculate vector mean
// [[Rcpp::export]]
double calculate_mean(NumericVector x) {
    double total = 0.0;
    for (int i = 0; i < x.size(); ++i) {
        total += x[i];
    }
    return total / x.size();
}
```

Bad Practice:

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double mean(NumericVector x){ // No spaces, poor naming
double t=0; // Single-letter variable
for(int i=0;i<x.size();i++) t+=x[i]; // No braces
return t/x.size();}
```

2. Rcpp Namespace Utility and Best Practices /d

Key Utilities:

- `Rcpp::NumericVector`, `Rcpp::DataFrame`: R data structures
- `Rcpp::stop()`: Exception handling
- `Rcpp::Rcout`: Output redirection

Best Practices:

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
NumericVector safe_divide(NumericVector a, NumericVector b) {
  if (a.size() != b.size()) {
    Rcpp::stop("Vectors must be same length");
  }

  NumericVector result(a.size());
  for (int i = 0; i < a.size(); ++i) {
    if (b[i] == 0) {
      Rcpp::warning("Division by zero at index %d", i);
      result[i] = NA_REAL;
    } else {
      result[i] = a[i] / b[i];
    }
  }
  return result;
}
```

3. RcppArmadillo and RcppEigen /d

RcppArmadillo:

- Linear algebra with MATLAB-like syntax

```
#include <RcppArmadillo.h>
// [[Rcpp::depends(RcppArmadillo)]]

// [[Rcpp::export]]
arma::mat arma_matrix_mult(arma::mat A, arma::mat B) {
  return A * B;
}
```

RcppEigen:

- Template-based linear algebra

```
#include <RcppEigen.h>
// [[Rcpp::depends(RcppEigen)]]

// [[Rcpp::export]]
Eigen::MatrixXd eigen_matrix_mult(Eigen::MatrixXd A, Eigen::MatrixXd B) {
  return A * B;
}
```

4. Integration with Tidyverse Packages

Approach: Return DataFrame for dplyr piping

```
// [[Rcpp::export]]
DataFrame process_data(DataFrame df) {
  NumericVector x = df["x"];
  NumericVector y = exp(x) / sum(exp(x));
  return DataFrame::create(_["x"] = x, _["y"] = y);
}
```

R Usage:

```
library(dplyr)
df %>%
  process_data() %>%
  filter(y > 0.1) %>%
  mutate(z = y * 2)
```

5. Integration with OS Libraries (GSL)

Implementation:

1. Add to DESCRIPTION:

```
SystemRequirements: GNU GSL
LinkingTo: Rcpp
```

2. Create src/Makevars:

```
PKG_LIBS = `$(R_HOME)/bin/Rscript -e "Rcpp:::LdFlags()"` -lgsl -lgslcblas
```

C++ Example:

```
#include <gsl/gsl_sf_bessel.h>
#include <Rcpp.h>
```

```
// [[Rcpp::export]]
NumericVector gsl_bessel_j0(NumericVector x) {
  NumericVector y(x.size());
  for(int i=0; i<x.size(); i++) {
    y[i] = gsl_sf_bessel_J0(x[i]);
  }
  return y;
}
```

6. Roxygen2 Documentation Best Practices

C++ File Example:

```
//' Compute Euclidean distance matrix
//'
//' @param x Numeric matrix (n x p)
//' @return n x n distance matrix
//' @examples
//' dist_matrix(as.matrix(iris[,1:2]))
//' @export
// [[Rcpp::export]]
NumericMatrix dist_matrix(NumericMatrix x) {
  // ... implementation ...
}
```

7. R and C++ Function Calls

Calling R Functions:

```
// [[Rcpp::export]]
NumericVector rnorm_rcpp(int n) {
  Function rnorm("rnorm");
  return rnorm(n, _["mean"] = 0, _["sd"] = 1);
}
```

Calling C++ Functions:

```
double internal_helper(double x) { return x*x; }

// [[Rcpp::export]]
NumericVector square_vec(NumericVector x) {
  NumericVector y(x.size());
```

```

    for(int i=0; i<x.size(); i++) {
        y[i] = internal_helper(x[i]);
    }
    return y;
}

```

8. Rebuild Operations

Commands:

```

# Full rebuild
R CMD INSTALL --preclean --clean .

# Partial rebuild (devtools)
devtools::load_all() # During development
devtools::document() # Update documentation

```

9. Unit Testing with testthat

Test File (tests/testthat/test-rcpp.R):

```

test_that("Matrix multiplication works", {
  A <- matrix(1:4, ncol=2)
  B <- matrix(c(1,0,0,1), ncol=2)
  expect_equal(arma_matrix_mult(A, B), A)
})

test_that("Error handling works", {
  expect_error(safe_divide(1:3, 1:2))
})

```

Supplemental Materials

1. C++ Source Skeleton

```

// mypackage/src/rcpp_functions.cpp

// Enable C++11
// [[Rcpp::plugins(cpp11)]]

// Dependencies

```

```

// [[Rcpp::depends(RcppArmadillo, RcppEigen)]]

#include <Rcpp.h>
#include <RcppArmadillo.h>
#include <RcppEigen.h>
#include <vector>
#include <gsl/gsl_math.h>

using namespace Rcpp;

//' @title Sample Function
//' @description Demonstrates multiple library usage
//' @param x NumericVector input
//' @return List with transformed outputs
//' @export
// [[Rcpp::export]]
List sample_function(NumericVector x) {
  // Rcpp operation
  double rcpp_mean = mean(x);

  // Armadillo operation
  arma::vec arma_x = as<arma::vec>(x);
  double arma_var = arma::var(arma_x);

  // Eigen operation
  Eigen::VectorXd eigen_x = as<Eigen::VectorXd>(x);
  double eigen_norm = eigen_x.norm();

  // GSL operation
  double gsl_result = gsl_hypot(3, 4); // 5.0

  return List::create(
    Named("mean") = rcpp_mean,
    Named("variance") = arma_var,
    Named("norm") = eigen_norm,
    Named("gsl_hypot") = gsl_result
  );
}

```

2. DESCRIPTION Requisites

Package: mypackage

Title: My R Package

Version: 0.1.0

```
Imports:
  Rcpp (≥ 1.0.7)
LinkingTo:
  Rcpp,
  RcppArmadillo,
  RcppEigen
SystemRequirements:
  GNU GSL (≥ 2.7),
  C++11
```

3. CI/CD Pipeline Templates

3.1 GitHub Action

GitHub Action (.github/workflows/rcpp-ci.yml):

```
name: R-CMD-Check

on: [push, pull_request]

jobs:
  build:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]

    steps:
      - uses: actions/checkout@v2

      - name: Set up R
        uses: r-lib/actions/setup-r@v1

      - name: Install dependencies
        run: |
          sudo apt-get install -y libgsl-dev # Linux only
          R -e "install.packages(c('devtools', 'rcmdcheck'))"
          R -e "devtools::install_deps(dependencies=TRUE)"

      - name: Build and check
        run: R CMD build . && R CMD check *.tar.gz --as-cran

      - name: Upload binary
        if: success() && matrix.os == 'ubuntu-latest'
```

```
env:
  FTP_USER: ${ secrets.FTP_USER }
  FTP_PASS: ${ secrets.FTP_PASS }
run: |
  curl -T mypackage*.tar.gz \
    -u $FTP_USER:$FTP_PASS \
    ftp://myftpserver.com/packages/
```

3.1 GitLab CI

GitLab CI (.gitlab-ci.yml):

```
image: rocker/r-ver:4.3.0

stages:
  - build
  - deploy

variables:
  GIT_SUBMODULE_STRATEGY: recursive

before_script:
  - apt-get update -qq && apt-get install -y libgsl-dev
  - R -e "install.packages(c('devtools', 'rcmdcheck'))"
  - R -e "devtools::install_deps()"

build:
  stage: build
  script:
    - R CMD build .
    - R CMD check *.tar.gz --as-cran

deploy:
  stage: deploy
  only:
    - main
  script:
    - curl -T mypackage*.tar.gz
      -u $FTP_USER:$FTP_PASSWORD
      ftp://myftpserver.com/packages/
```

This guide provides a comprehensive overview of Rcpp best practices, CRAN compliance, and modern development workflows. Always validate with `R CMD check --as-cran` before submission.

A:1 (Kimi)

^ *Comprehensive Guide to Rcpp Usage in R Packages*

1. C++11 Rcpp Sources Coding Style /k

Recommended Coding Style /k

The recommended coding style for C++11 Rcpp sources follows the [Google C++ Style Guide](#) with R-specific adaptations:

Good Practices

// Good: Clear naming, proper spacing, const correctness

```
namespace mypackage {  
  
double compute_sum(const Rcpp::NumericVector& x) {  
    double total = 0.0;  
    for (size_t i = 0; i < x.size(); ++i) {  
        total += x[i];  
    }  
    return total;  
}  
  
} // namespace mypackage
```

Bad Practices

// Bad: Poor naming, magic numbers, no const correctness

```
double computeSum(Rcpp::NumericVector x){  
double total=0;  
for(int i=0;i<x.size();i++){  
total+=x[i];  
}  
return total;  
}
```

Key Style Guidelines

- **Naming:** Use snake_case for variables/functions, PascalCase for classes
- **Indentation:** 2 spaces (no tabs)

- **Line length:** Maximum 80 characters
- **Headers:** Include guards using `#pragma once`
- **Namespaces:** Always wrap code in a package-specific namespace

2. Rcpp Namespace Utility and Best Practices /k

Namespace Utility

The Rcpp namespace provides:

- Seamless R/C++ data type conversion
- Memory management through R's garbage collector
- Exception handling bridging R errors and C++ exceptions

Best Practices /k

Basic I/O /k

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
List safe_io_example(NumericVector x) {
  // Input validation
  if (x.size() == 0) {
    Rcpp::stop("Input vector cannot be empty");
  }

  // Rcpp::Rcout for C++ output to R console
  Rcpp::Rcout << "Processing vector of length " << x.size() << std::endl;

  return List::create(
    Named("mean") = Rcpp::mean(x),
    Named("sum") = Rcpp::sum(x)
  );
}
```

Memory Management /k

```
// [[Rcpp::export]]
IntegerVector memory_efficient_resize(IntegerVector x, int new_size) {
  // Rcpp automatically manages memory through R's GC
  IntegerVector result(new_size);

  // Efficient copy with bounds checking
  std::copy(x.begin(),
    x.begin() + std::min(x.size(), static_cast<R_xlen_t>(new_size)),
    result.begin());
}
```

```

        result.begin());

    return result;
}

```

Exception Handling /k

```

// [[Rcpp::export]]
double safe_division(double a, double b) {
    try {
        if (b == 0.0) {
            throw std::runtime_error("Division by zero");
        }
        return a / b;
    } catch (const std::exception& e) {
        Rcpp::stop("Error in division: " + std::string(e.what()));
    }
}

```

3. RcppArmadillo and RcppEigen /k

RcppArmadillo /k

Purpose: Provides seamless integration between R and Armadillo C++ linear algebra library.

Benefits:

- High-performance linear algebra operations
- MATLAB-like syntax
- Automatic memory management

Sample Usage /k

```

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::export]]
arma::mat arma_multiply(const arma::mat& A, const arma::mat& B) {
    // Matrix multiplication
    return A * B;
}

// [[Rcpp::export]]
List arma_eigen(const arma::mat& X) {
    arma::vec eigenvals;
    arma::mat eigenvecs;
}

```

```

bool success = arma::eig_sym(eigenvals, eigenvecs, X);

return List::create(
  Named("values") = eigenvals,
  Named("vectors") = eigenvecs,
  Named("success") = success
);
}

```

RcppEigen /k

Purpose: Integration with Eigen C++ template library for linear algebra.

Benefits:

- Expression templates for optimized performance
- Header-only library
- Advanced linear algebra operations

Sample Usage /k

```

// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>

// [[Rcpp::export]]
Eigen::MatrixXd eigen_solve(const Eigen::Map<Eigen::MatrixXd>& A,
                           const Eigen::Map<Eigen::VectorXd>& b) {
  // Solve linear system Ax = b
  return A.colPivHouseholderQr().solve(b);
}

// [[Rcpp::export]]
double eigen_determinant(const Eigen::Map<Eigen::MatrixXd>& M) {
  return M.determinant();
}

```

4. Integration with Tidyverse Packages

Integration with dplyr

```

// [[Rcpp::export]]
DataFrame rcpp_dplyr_grouped_sum(DataFrame df, String group_col, String value_col) {
  // Extract columns
  CharacterVector groups = df[group_col];
  NumericVector values = df[value_col];

  // Create map for grouped sums

```

```

std::unordered_map<std::string, double> group_sums;

for (int i = 0; i < groups.size(); ++i) {
  if (CharacterVector::is_na(groups[i]) || NumericVector::is_na(values[i])) continue;

  std::string group = Rcpp::as<std::string>(groups[i]);
  group_sums[group] += values[i];
}

// Convert back to R vectors
CharacterVector out_groups(group_sums.size());
NumericVector out_sums(group_sums.size());

size_t idx = 0;
for (const auto& pair : group_sums) {
  out_groups[idx] = pair.first;
  out_sums[idx] = pair.second;
  ++idx;
}

return DataFrame::create(
  Named("group") = out_groups,
  Named("sum") = out_sums
);
}

```

R Usage Example /k

```

library(dplyr)
library(Rcpp)

# Assuming the above C++ function is compiled
result <- rcpp_dplyr_grouped_sum(mtcars, "cyl", "mpg")

```

5. Integration with Standard OS Libraries (GSL)

GSL Integration

```

// [[Rcpp::depends(RcppGSL)]]
#include <RcppGSL.h>
#include <gsl/gsl_sf_gamma.h>
#include <gsl/gsl_rng.h>
#include <gsl/gsl_randist.h>

// [[Rcpp::export]]

```

```

NumericVector gsl_gamma_values(NumericVector x) {
  int n = x.size();
  NumericVector result(n);

  for (int i = 0; i < n; ++i) {
    result[i] = gsl_sf_gamma(x[i]);
  }

  return result;
}

// [[Rcpp::export]]
NumericVector gsl_rnorm(int n, double mu, double sigma) {
  gsl_rng* r = gsl_rng_alloc(gsl_rng_mt19937);
  gsl_rng_set(r, std::time(nullptr));

  NumericVector result(n);
  for (int i = 0; i < n; ++i) {
    result[i] = mu + gsl_ran_gaussian(r, sigma);
  }

  gsl_rng_free(r);
  return result;
}

```

Package Requirements for GSL

In DESCRIPTION:

SystemRequirements: GNU GSL

LinkingTo: RcppGSL

In Makevars:

PKG_CPPFLAGS = \$(GSL_CFLAGS)

PKG_LIBS = \$(GSL_LIBS)

6. Best Practices for Roxygen2 Comments

```

/// Compute the Euclidean distance between two vectors
///
/// This function calculates the Euclidean distance between two numeric vectors
/// of equal length.
///
/// @param x A numeric vector
/// @param y A numeric vector of same length as x

```

```

//' @return The Euclidean distance between x and y
//' @examples
//' x <- c(1, 2, 3)
//' y <- c(4, 5, 6)
//' euclidean_distance(x, y)
//' @export
// [[Rcpp::export]]
double euclidean_distance(const NumericVector& x, const NumericVector& y) {
  if (x.size() != y.size()) {
    Rcpp::stop("Vectors must be of equal length");
  }

  double sum = 0.0;
  for (int i = 0; i < x.size(); ++i) {
    sum += pow(x[i] - y[i], 2);
  }

  return sqrt(sum);
}

```

7. R and C++ Function Calls

Calling R Functions from Rcpp

```

// [[Rcpp::export]]
List call_r_function(NumericVector x) {
  // Get R's sd function
  Rcpp::Environment stats("package:stats");
  Rcpp::Function sd_fun = stats["sd"];

  // Call the function
  double std_dev = Rcpp::as<double>(sd_fun(x));

  // Call another C++ function
  double mean_val = Rcpp::mean(x);

  return List::create(
    Named("mean") = mean_val,
    Named("sd") = std_dev
  );
}

```

Calling C++ Functions from Rcpp

```
// Helper function (not exported)
double internal_helper(const NumericVector& x) {
  return Rcpp::sum(x * x);
}

// [[Rcpp::export]]
double use_helper(NumericVector x) {
  return internal_helper(x);
}
```

8. Rebuild Operations /k

Rebuilding Rcpp Functions

```
# Method 1: Direct compilation
Rcpp::sourceCpp("src/my_functions.cpp")

# Method 2: Package development
devtools::load_all() # Recompiles and loads

# Method 3: Standard R commands
R CMD build mypackage
R CMD check mypackage.tar.gz

# Method 4: Using devtools
devtools::check()
devtools::build()
devtools::install()
```

9. Code Unit Testing with testthat

Test File Structure (tests/testthat/test-rcpp.R)

```
test_that("Euclidean distance works correctly", {
  # Test basic functionality
  x <- c(0, 0)
  y <- c(3, 4)
  expect_equal(euclidean_distance(x, y), 5)

  # Test with negative values
  x <- c(-1, -1)
  y <- c(2, 3)
  expect_equal(euclidean_distance(x, y), 5)
```

```

    # Test error handling
    expect_error(euclidean_distance(c(1, 2), c(1, 2, 3)))
  })

test_that("Gamma function integration works", {
  # Test GSL integration
  x <- seq(1, 5, by = 0.5)
  result <- gsl_gamma_values(x)
  expected <- gamma(x)
  expect_equal(result, expected, tolerance = 1e-10)
})

```

Full C++ Source Skeleton

```

// #pragma once
#ifndef MYPACKAGE_UTILS_HPP
#define MYPACKAGE_UTILS_HPP

// Enable C++11
// [[Rcpp::plugins(cpp11)]]

// Rcpp headers
#include <Rcpp.h>

// Additional Rcpp packages
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>

// [[Rcpp::depends(RcppGSL)]]
#include <RcppGSL.h>

// Standard library headers
#include <vector>
#include <algorithm>
#include <cmath>
#include <memory>
#include <unordered_map>

// GSL headers
#include <gsl/gsl_sf_gamma.h>
#include <gsl/gsl_rng.h>

```

```

#include <gsl/gsl_randist.h>
#include <gsl/gsl_statistics.h>

// Package namespace
namespace mypackage {

// Forward declarations
double internal_helper(const Rcpp::NumericVector& x);
arma::mat arma_operations(const arma::mat& A);

// Main exported function
//' Main computation function
//'
//' @param x Input matrix
//' @param y Input vector
//' @return List containing results
// [[Rcpp::export]]
Rcpp::List main_computation(const Rcpp::NumericMatrix& x,
                             const Rcpp::NumericVector& y) {
  try {
    // Validate inputs
    if (x.nrow() != y.size()) {
      Rcpp::stop("Matrix rows must match vector length");
    }

    // Convert to Armadillo
    arma::mat X = Rcpp::as<arma::mat>(x);
    arma::vec Y = Rcpp::as<arma::vec>(y);

    // Perform operations
    arma::mat XtX = X.t() * X;
    arma::vec beta = arma::solve(XtX, X.t() * Y);

    // Eigen operations
    Eigen::Map<Eigen::MatrixXd> eigen_X(Rcpp::as<Eigen::Map<Eigen::MatrixXd>>(x));
    Eigen::VectorXd eigen_Y = Rcpp::as<Eigen::VectorXd>(y);

    // Return results
    return Rcpp::List::create(
      Rcpp::Named("beta") = beta,
      Rcpp::Named("determinant") = arma::det(XtX),
      Rcpp::Named("eigen_values") = Rcpp::eigen_values(XtX)
    );
  } catch (const std::exception& e) {

```

```

    Rcpp::stop("Error in computation: " + std::string(e.what()));
  }
}

} // namespace mypackage

#endif // MYPACKAGE_UTILS_HPP

```

Roxygen2 Documentation and DESCRIPTION Requisites

DESCRIPTION File /k

```

Package: mypackage
Type: Package
Title: Advanced Rcpp Package with Multiple Dependencies
Version: 1.0.0
Authors@R: c(
  person("John", "Doe", email = "john@example.com", role = c("aut", "cre"))
)
Description: A comprehensive Rcpp package demonstrating integration with
  Armadillo, Eigen, and GSL libraries.
License: GPL (≥ 3)
Encoding: UTF-8
LazyData: true
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.2.3
Depends:
  R (≥ 4.0.0)
Imports:
  Rcpp (≥ 1.0.10),
  dplyr
LinkingTo:
  Rcpp,
  RcppArmadillo,
  RcppEigen,
  RcppGSL
SystemRequirements: GNU GSL, C++11
Suggests:
  testthat (≥ 3.0.0)
Config/testthat/edition: 3

```

NAMESPACE File (auto-generated) /k

```
# Generated by roxygen2: do not edit by hand
```

```
exportPattern("^[:alpha:]+$")
importFrom(Rcpp, evalCpp)
useDynLib(mypackage, .registration = TRUE)
```

GitHub/GitLab Packaging Action Pipeline

GitHub Actions (.github/workflows/R-CMD-check.yaml)

```
name: R-CMD-check

on:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]

jobs:
  R-CMD-check:
    runs-on: ${{ matrix.config.os }}

    name: ${{ matrix.config.os }} (${{ matrix.config.r }})

    strategy:
      fail-fast: false
      matrix:
        config:
          - {os: ubuntu-latest,  r: 'release'}
          - {os: macOS-latest,   r: 'release'}
          - {os: windows-latest, r: 'release'}

    env:
      GITHUB_PAT: ${{ secrets.GITHUB_TOKEN }}
      R_KEEP_PKG_SOURCE: yes
      FTP_USER: ${{ secrets.FTP_USER }}
      FTP_PASSWORD: ${{ secrets.FTP_PASSWORD }}
      FTP_SERVER: ftp.example.com

    steps:
      - uses: actions/checkout@v3

      - uses: r-lib/actions/setup-pandoc@v2

      - uses: r-lib/actions/setup-r@v2
        with:
          r-version: ${{ matrix.config.r }}
```

```

    http-user-agent: ${ matrix.config.http-user-agent }
    use-public-rspm: true

- name: Install system dependencies (Ubuntu)
  if: runner.os == 'Linux'
  run: |
    sudo apt-get update -y
    sudo apt-get install -y libgsl-dev

- name: Install system dependencies (macOS)
  if: runner.os == 'macOS'
  run: |
    brew install gsl

- name: Install dependencies
  uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: any::rcmdcheck

- name: Check
  uses: r-lib/actions/check-r-package@v2
  with:
    upload-snapshots: true

- name: Build binary package
  run: |
    R CMD build .
    PKG_FILE=$(ls -t *.tar.gz | head -n 1)
    echo "PKG_FILE=$PKG_FILE" >> $GITHUB_ENV

- name: Upload to FTP (if successful)
  if: success() && matrix.config.os == 'ubuntu-latest'
  run: |
    sudo apt-get install -y lftp
    lftp -u $FTP_USER,$FTP_PASSWORD $FTP_SERVER <<EOF
    set ftp:ssl-allow no
    cd /packages
    put ${ env.PKG_FILE }
    bye
    EOF

```

GitLab CI (.gitlab-ci.yml)

```

stages:
  - test

```

```
- build
- deploy

variables:
  _R_CHECK_FORCE_SUGGESTS_: "FALSE"
  FTP_USER: "$FTP_USER"
  FTP_PASSWORD: "$FTP_PASSWORD"
  FTP_SERVER: "ftp.example.com"

test:
  stage: test
  image: rocker/r-base:latest
  before_script:
    - apt-get update -y
    - apt-get install -y libgsl-dev libcurl4-openssl-dev libssl-dev
  script:
    - R -e "install.packages(c('devtools', 'testthat', 'Rcpp', 'RcppArmadillo', 'RcppEigen', 'RcppParallel'))"
    - R -e "devtools::check()"

build:
  stage: build
  image: rocker/r-base:latest
  only:
    - main
  script:
    - apt-get update -y
    - apt-get install -y libgsl-dev
    - R CMD build .
    - PKG_FILE=$(ls -t *.tar.gz | head -n 1)
    - echo "Built $PKG_FILE"
  artifacts:
    paths:
      - "*.tar.gz"

deploy:
  stage: deploy
  image: alpine:latest
  only:
    - main
  dependencies:
    - build
  script:
    - apk add --no-cache lftp
    - PKG_FILE=$(ls -t *.tar.gz | head -n 1)
    - lftp -u $FTP_USER,$FTP_PASSWORD $FTP_SERVER -e "set ftp:ssl-allow no; cd /packages; put $PKG_FILE"
```

This comprehensive guide provides all necessary information for developing, documenting, testing, and deploying R packages with Rcpp, including integration with major C++ libraries and continuous integration workflows.